Week 2 · Surge Speed Control

USV Guidance, Navigation and Control (Graduate) · Department of Autonomous Vehicle System Engineering, Chungnam National University | revised 2026-09-04

★ Reference material — read this first

The five courses below are **taught by the instructor of this course** and are the assumed background for it. They run from the fundamentals down to the graduate material, so start wherever the gap is. Every frame, symbol and derivation used here is developed in them at length; anyone whose prerequisites are thin should work through them first, then return.

#	Course	Level	Lang.	Video	Slides and code
1	**Control Engineering** (제어공학) — the foundation: transfer functions, feedback, stability, root locus, PID	undergraduate	KO	playlist	drive
2	**Control System Design** (제어시스템설계) — design rather than analysis: specifications, loop shaping, discrete implementation	undergraduate	KO	playlist	drive
3	**Advanced Control Engineering** (제어공학특론) — reference frames, the 6-DOF equation of motion, rotation matrices and Euler angles, linearisation and trim, vehicle control design	graduate	EN	playlist	drive
4	**Sensor Signal Processing and Fusion** (센서신호처리 및 융합) — sensor models, noise, estimation and multi-sensor fusion	graduate	EN	playlist	drive
5	**Capstone Design** (캡스톤디자인) — a vehicle project carried end to end	undergraduate	KO	playlist	drive

Not fluent in MATLAB or Simulink yet? Do these before Part 2. Every laboratory in this course is Simulink, and the Onramp courses are free and take a few hours each.

Tool	Where to start
MATLAB	MATLAB Onramp · Core MATLAB Skills
Simulink	Simulink Onramp · instructor's Simulink lectures part 1 · part 2

- **Course**: USV Guidance, Navigation and Control (Graduate)
- **Department**: Autonomous Vehicle System Engineering, Chungnam National University
- **This week**: ① the surge axis as a first-order, type 0 plant identified from the plant itself ② why proportional control leaves an error and integral action removes it ③ what an actuator limit does to an integrator, and three ways of surviving it

★ Prerequisites from the previous week

- From Week 1: $\tau_u = 1.1025$ s and $K_u = 0.012894$ (m/s)/N, and the propeller curve $T = k\,n|n|$.
- Two numbers, and nothing else, are carried in from **Appendix A1**: the surge force this vessel can produce lies in $X \in [-133.42,\ 239.36]$ N. That limit is what causes the windup of §2-6. Appendix A1 derives it; a reader who takes the two numbers on trust loses nothing this week.

Learning Outcomes

Upon completion of this week, the learner is able to:

1. Identify the DC gain and time constant of the surge axis from a step response, and state the discrepancy between the measurement and the first-order model.
2. Predict, from the final value theorem, the steady-state error of a proportional controller on a type 0 plant, and explain why no gain removes it.
3. Choose K_p and K_i to place the closed-loop poles at a specified ζ and ω_n, and explain why the resulting overshoot exceeds the value tabulated for that ζ.
4. State where the derivative term enters the equation of motion of a velocity loop, and predict its effect on the damping ratio before running the simulation.
5. Explain integrator windup as a consequence of actuator saturation rather than of integral action, and implement clamping and back-calculation.
6. Report a settling or recovery time with the tolerance band and the averaging window stated.
7. Build a PID controller from elementary blocks and from Simulink's PID Controller block, and state where the two differ and why.
8. Explain why a PID never differentiates directly, write the pseudo-derivative $Ns/(s+N)$ as a filter and a subtraction, and choose N from the closed-loop bandwidth rather than by making it large.

Prerequisites and Setup

Item	Requirement
MATLAB	R2024b, with Simulink
Toolboxes	Control System Toolbox, for `tf`, `feedback` and `stepinfo` in the analysis script
MSS	`Tools/MSS`, added by the setup script
Course folder	`GradCourse/lectures/W02_simulink`
Expected duration	60 min theory, 90 min laboratory

Part 1 · Theory

2-1. The plant, reduced to two numbers

Where the surge equation comes from

- The controller of this week acts on one scalar equation. That equation is not an assumption — it is the first row of the 3-DOF model of §1-7, and this section derives it so that the terms thrown away can be named and later reclaimed.
- Start from the horizontal-plane model, $\boldsymbol{\nu} = [u\ v\ r]^\top$ (Fossen, *Handbook*, 2nd ed., §7.3):

$$\mathbf{M}\dot{\boldsymbol{\nu}} + \mathbf{C}(\boldsymbol{\nu})\boldsymbol{\nu} + \mathbf{D}(\boldsymbol{\nu})\boldsymbol{\nu} = \boldsymbol{\tau}$$

- For a vessel symmetric about its centreline, with the body origin on that centreline and the centre of gravity offset by x_g along x_b:

$$\mathbf{M} = \begin{bmatrix} m - X_{\dot{u}} & 0 & 0 \\ 0 & m - Y_{\dot{v}} & mx_g - Y_{\dot{r}} \\ 0 & mx_g - N_{\dot{v}} & I_z - N_{\dot{r}} \end{bmatrix}$$

- For the Otter, with the payload this course uses, that matrix is

$$\mathbf{M} = \begin{bmatrix} 85.5000 & 0 & 0 \\ 0 & 162.5000 & 12.2500 \\ 0 & 12.2500 & 42.6515 \end{bmatrix} \quad \text{kg, kg·m, kg·m}^2$$

> **★ The surge row of $\mathbf{M}$ is already decoupled, and that is structural**
>
> The zeros in the first row and column are **exactly zero**, not small. They follow from **port–starboard symmetry**: a hull that is mirror-symmetric cannot produce a sway force or a yaw moment when it is accelerated purely forward. Everything below concerns $\mathbf{C}$ and $\mathbf{D}$, because $\mathbf{M}$ has already given the surge axis to itself.
>
> The off-diagonal $M_{26} = 12.25$ kg·m couples sway and yaw, and it is the reason a turn produces sway. It never touches surge.

- The matrix above is not transcribed. It is rebuilt from `otter.m` lines 55–120 and printed by

```
verify_constants
```

which also checks the sixteen coefficients this course quotes and reports `every quoted value agrees with otter.m to its printed precision`.

- The Coriolis matrix contributes to surge through the rigid-body and added-mass parts. Taking the first row of $\mathbf{C}_{RB}(\boldsymbol{\nu})\boldsymbol{\nu} + \mathbf{C}_A(\boldsymbol{\nu})\boldsymbol{\nu}$ and collecting terms gives the standard manoeuvring form:

$$\underbrace{(m - X_{\dot{u}})\,\dot{u}}_{\text{acceleration}} - \underbrace{(m - Y_{\dot{v}})\,vr}_{\text{Coriolis, sway}\times\text{yaw}} - \underbrace{(mx_g - Y_{\dot{r}})\,r^2}_{\text{centripetal}}$$
$$= \underbrace{X}_{\text{thrust}} + \underbrace{X_u u}_{\text{damping, } X_u<0}$$

What is dropped, and why it is exact here

- Two terms stand between the full row and the scalar plant, and **both carry a factor of r**:

Term	Dropped because
$(m - Y_{\dot v})\, vr$	proportional to r
$(m x_g - Y_{\dot r})\, r^2$	proportional to r^2

- In this week's experiment r is not merely small. It is **identically zero**, and the reason is the actuator, not the hull:

$$\boldsymbol{\tau} = \mathbf{B}\mathbf{f}, \qquad \mathbf{B} = \begin{bmatrix} 1 & 1 \\ 0 & 0 \\ y_p & -y_p \end{bmatrix}, \qquad \mathbf{f} = \begin{bmatrix} T \\ T \end{bmatrix} \implies \boldsymbol{\tau} = \begin{bmatrix} 2T \\ 0 \\ 0 \end{bmatrix}$$

- The speed controller splits its demand **equally** between the two propellers, so $N = y_p(T - T) = 0$ exactly. With $N = 0$ and $r(0) = 0$ the yaw equation gives $r \equiv 0$, and with $r \equiv 0$ the sway equation gives $v \equiv 0$.
- Therefore both discarded terms are **exactly zero for the whole run**, not small. This is the rare case where a 1-DOF reduction is not an approximation.

$$\boxed{(m - X_{\dot u})\, \dot u = X + X_u u} \qquad \iff \qquad M_{11}\, \dot u = X + X_u u$$

$$M_{11} = (m + m_p) - X_{\dot u} = 85.50 \text{ kg}$$

> **i When the dropped terms come back**
>
> The moment a heading command is added, $r \neq 0$ and vr reappears in the surge equation — the vessel slows in a turn without any change in thrust. Week 1 measured exactly this: surge fell from 1.0286 to 1.0218 m/s in the turns, a loss of 0.7%, produced entirely by the term dropped above. Week 3 controls heading and Week 5 runs both loops at once.

- Taking Laplace transforms with $u(0) = 0$ gives a first-order lag:

$$\frac{u(s)}{X(s)} = \frac{K_u}{\tau_u s + 1}, \qquad K_u = \frac{1}{|X_u|}, \qquad \tau_u = \frac{M_{11}}{|X_u|}.$$

Symbol	Quantity	Value / source
M_{11}	surge mass including added mass	85.50 kg, `otter.m`
X_u	linear surge damping	-77.5544 N per m/s, $-24.4 g/U_{\max}$
K_u	DC gain	0.012894 (m/s)/N
τ_u	time constant	1.1025 s

> **★ The plant has no free integrator**
>
> There is no $1/s$ anywhere in $u(s)/X(s)$. The loop is therefore **type 0**, and every consequence of §2-2 follows from that single structural fact rather than from any numerical value.

2-2. Proportional control, and the error it cannot remove

- With $X = K_p(u_d - u)$ the closed loop is

$$\frac{u(s)}{u_d(s)} = \frac{K_p K_u}{\tau_u s + 1 + K_p K_u},$$

- and the final value theorem gives the steady state directly:

$$\boxed{\frac{u_{ss}}{u_d} = \frac{K_p K_u}{1 + K_p K_u} \qquad \Longrightarrow \qquad \frac{e_{ss}}{u_d} = \frac{1}{1 + K_p K_u}}$$

- The error is never zero for finite K_p. The physical reason is more useful than the algebraic one:

> **i The error is what produces the force**
>
> Holding a steady speed requires a steady force, $X_{ss} = |X_u|\, u_{ss}$, because the damping never stops. A proportional controller produces force **only** from error. Removing the error would remove the force that sustains the speed, so the error is not a defect of the controller — it is the controller's only means of doing its job.

- Raising the gain shrinks the error but never removes it, and it does so at a cost. The gain also multiplies measurement noise and pushes the demanded force toward the saturation limits of §2-6.

2-3. Integral action, and the zero nobody placed

- Adding $K_i \int e\,\mathrm{d}t$ supplies the steady force without steady error. The closed loop becomes second order:

$$\frac{u(s)}{u_d(s)} = \frac{K_u\,(K_p s + K_i)}{\tau_u s^2 + (1 + K_u K_p)s + K_u K_i}.$$

- Matching the denominator to $s^2 + 2\zeta\omega_n s + \omega_n^2$ gives the design equations used in `W02_0_setup.m`:

$$\omega_n = \sqrt{\frac{K_u K_i}{\tau_u}}, \qquad \zeta = \frac{1 + K_u K_p}{2\sqrt{\tau_u K_u K_i}}.$$

- Inverting them for a specified pair:

$$K_i = \frac{\omega_n^2\,\tau_u}{K_u}, \qquad K_p = \frac{2\zeta\sqrt{\tau_u K_u K_i} - 1}{K_u}.$$

- For $\zeta = 0.7$ and $\omega_n = 1.5$ rad/s this gives $K_p = 102.00$ and $K_i = 192.38$.

> **✕ The tabulated overshoot for $\zeta = 0.7$ does not apply**
>
> The numerator of the closed loop is $K_u(K_p s + K_i)$, which is a **zero** at $s = -K_i/K_p = -1.886$. The standard relation $M_p = \exp\left(-\pi\zeta/\sqrt{1-\zeta^2}\right)$ is derived for a second-order system with **no** zero. A PI controller always adds one, and it always lands at $-K_i/K_p$, which for any sensible design sits close to the poles. The measured overshoot is 8.1% against the tabulated 4.6% — a factor of 1.8. The damping ratio was designed correctly; the prediction made from it was not.

2-4. Where the derivative term actually goes

- The derivative is taken on the **measurement**, not on the error:

$$X = K_p e + K_i \int e\,\mathrm{d}t - K_d\,\frac{Ns}{s+N}\,u.$$

- Two reasons, and only the second is about this week.

Choice	Reason
on the measurement, not the error	a step in u_d differentiates to an impulse; the measurement never steps
filtered by $Ns/(s+N)$	pure differentiation has unbounded high-frequency gain and amplifies sensor noise without limit

The pseudo-derivative

- The second row deserves more than a line, because **no working PID contains a differentiator.** What it contains is an approximation to one, and that approximation has a name.

$$\underbrace{s}_{\text{ideal}} \qquad \longrightarrow \qquad \underbrace{\frac{Ns}{s+N}}_{\text{pseudo-derivative}}$$

- Compare the two by magnitude alone:

$$|j\omega| = \omega \qquad \text{against} \qquad \left|\frac{Nj\omega}{j\omega + N}\right| = \frac{N\omega}{\sqrt{\omega^2 + N^2}}$$

ω	ideal	pseudo
$\omega \ll N$	ω	$\approx \omega$ — the two agree
$\omega = N$	N	$N/\sqrt{2}$
$\omega \gg N$	$\omega \to \infty$	$\to N$, **flat**

> **★ Differentiation multiplies every component by its own frequency**
>
> That is the whole problem in one sentence. The fastest thing in any real measurement is the sensor noise, so differentiation seeks out the least meaningful part of the signal and multiplies it by the largest number in the problem. The pseudo-derivative is the same operator with its gain capped at N, and N is therefore the knob that decides **how much of the noise reaches the actuator**.

- Written as a state, the pseudo-derivative is a first-order lag on the signal, subtracted from it:

$$\frac{Ns}{s+N} = N\left(1 - \frac{N}{s+N}\right) \qquad \Longleftrightarrow \qquad \dot{x}_f = N\,(y - x_f), \quad d = N\,(y - x_f)$$

- So it costs one state and no differentiation at all. **A low-pass filter and a subtraction are doing the work of a derivative**, which is why it is implementable in fixed point on a microcontroller and why an ideal derivative is not.

Choosing N	What happens
N too small	the filter lags, the derivative arrives late, and the damping it was added for is lost
N too large	the response stops improving — the pole is far outside the loop bandwidth — while the noise keeps growing

- There is a common belief that large N is the safe default because it is "closer to a true derivative". Section I measures it and finds the opposite: above a certain N the response is unchanged to within 0.3 points of overshoot while the actuator noise grows by a factor of three.
- The rule that follows: **place N just above the closed-loop bandwidth, and no higher.**

i The name

Textbooks call $Ns/(s+N)$ the *filtered*, *approximate*, *practical* or *pseudo*-derivative interchangeably. Åström and Hägglund parameterise it as $T_d s/(1+(T_d/N)s)$, where $T_d = K_d/K_p$ and $N \in [8, 20]$ is dimensionless. That N and the N used here are related by $N_{\text{here}} = N_{\text{Åström}}/T_d$, so the two conventions must never be mixed in one derivation.

- Now substitute into the equation of motion. Ignoring the filter, $-K_d\dot{u}$ moves to the left-hand side:

$$(M_{11} + K_d)\dot{u} = X_{\text{PI}} + X_u u.$$

★ On a velocity loop the derivative term is a mass, not a damper

The controlled variable is a velocity, so its derivative is an **acceleration**, and K_d enters the equation of motion in exactly the place occupied by the mass. The effective time constant becomes $\tau_{\text{eff}} = \tau_u + K_u K_d$, so

$$\omega_n = \sqrt{\frac{K_u K_i}{\tau_u + K_u K_d}} \downarrow, \qquad \zeta = \frac{1 + K_u K_p}{2\sqrt{(\tau_u + K_u K_d) K_u K_i}} \downarrow.$$

Adding derivative action to this loop makes it **less** damped, not more. Section E measures it.

- This is a property of the **axis**, not of PID. Week 3 controls a heading, whose derivative is a rate rather than an acceleration, and there the same term supplies genuine damping. The lesson is to substitute the control law into the equation of motion before assuming what a term does.

2-5. Allocation, in its simplest form

- The controller demands a force. The propellers accept a shaft speed. Inverting $T = k\,n|n|$ with the demand split equally:

$$T = \frac{X_{\text{cmd}}}{2}, \qquad n = \text{sign}(T)\sqrt{\frac{|T|}{k}}, \qquad k = \begin{cases} k_{\text{pos}}, & T \geq 0 \\ k_{\text{neg}}, & T < 0 \end{cases}$$

- then saturate n and compute what the propellers **actually** deliver:

$$X_{\text{sat}} = 2\,k\,n_{\text{sat}}|n_{\text{sat}}|.$$

- The equal split is forced: a pure surge demand contains nothing that distinguishes the two propellers. Week 4 treats the case where the demand does distinguish them and the split is no longer obvious.

⚠ X_{sat} must leave the allocation block

Without it the controller has no way of knowing that the actuator has stopped following the demand. Every anti-windup scheme in §2-6 is built on the difference $X_{\text{cmd}} - X_{\text{sat}}$, and a block that does not report what it delivered makes all of them impossible.

2-6. Windup is a saturation problem

- The attainable set of §A1-6 caps the surge force at $X_{\max} = 24.4g = 239.364$ N. Combined with $X_u = -24.4g/U_{\max}$ this produces an exact and rather elegant limit:

$$u_{\max} = \frac{X_{\max}}{|X_u|} = \frac{24.4g}{24.4g/U_{\max}} = U_{\max} = 3.0864 \text{ m/s}.$$

- The saturation limit and the damping coefficient are not independent numbers in `otter.m`. They are chosen so that full ahead delivers exactly the design speed of six knots. Astern, $X_{\min} = -13.6g$ gives $u_{\min} = -1.7203$ m/s.
- Ask for more than $u_{\max}$ and the error cannot be driven to zero. The integrator, which knows nothing of this, keeps integrating.

Stage	What happens
demand exceeds the limit	X_{sat} stops following X_{cmd}; the loop is open
the integrator continues	its state grows without bound while the error stays positive
the setpoint returns to a reachable value	the error changes sign, but the stored charge must be integrated back out before X_{cmd} re-enters the attainable set
meanwhile	the vessel does not respond at all

The principle, in one line

- Write the controller output and what the actuator actually delivers as two separate signals:

$$X_{\text{cmd}} = K_p e + I, \qquad X_{\text{sat}} = \text{sat}(X_{\text{cmd}}), \qquad \dot{I} = K_i e.$$

- Everything goes wrong at the third equation. $\dot{I}$ is written in terms of e, and e is the error of a loop that is **no longer closed**. Both remedies do the same thing: they make $\dot{I}$ depend on the saturation as well.

$$\boxed{\dot{I} = K_i e \;-\; \underbrace{f(X_{\text{cmd}} - X_{\text{sat}})}_{\text{zero whenever the actuator is following}}}$$

- The bracketed term vanishes whenever $X_{\text{cmd}} = X_{\text{sat}}$, so **an unsaturated loop is unaffected by any anti-windup scheme**. That is the property both schemes must have, and it is what makes them safe to switch on permanently.

Two remedies

Clamping, or conditional integration — freeze the integrator while the actuator is saturated *and* the error would drive it further in:

$$\dot{I} = \begin{cases} 0, & |X_{\text{cmd}} - X_{\text{sat}}| > 0 \ \text{ and } \ \text{sign}(e) = \text{sign}(X_{\text{cmd}} - X_{\text{sat}}) \\ K_i e, & \text{otherwise} \end{cases}$$

Back-calculation — feed the excess back into the integrator through a gain:

$$\dot{I} = K_i e - K_{\text{aw}}\,(X_{\text{cmd}} - X_{\text{sat}}), \qquad K_{\text{aw}} = \frac{1}{\tau_u}.$$

- Back-calculation has a fixed point worth naming. While the actuator is saturated, $\dot{I} \to 0$ requires

$$I \to \frac{K_i}{K_{\text{aw}}} e + X_{\text{sat}} - K_p e,$$

- which is the integrator value that would make the demand equal to what the actuator can give. The scheme does not merely stop the integrator; it **steers it to the boundary**.

	Clamping	Back-calculation
mechanism	logical, the integrator is switched off	continuous, the integrator is pulled to the boundary
tuning	none	one gain, K_{aw}
behaviour at the boundary	discontinuous	smooth
where the integrator ends up	wherever it was when saturation began	at the value that makes $X_{\mathrm{cmd}} = X_{\mathrm{sat}}$
large-gain limit	—	**not** clamping; §H measures the difference

> **i Neither is a fix for the integrator**
>
> Both schemes exist because the loop was opened by the actuator. The correct engineering response to persistent saturation is a reference the vessel can actually follow — which is what Week 7's reference model provides. Anti-windup limits the damage; it does not make an unreachable setpoint reachable.

2-7. The same controller, written twice

- Simulink ships a **PID Controller** block that contains everything §2-3 to §2-6 describes: three terms, a filtered derivative, an output limit, and the same two anti-windup schemes as dialog options.
- Both are in the model, and `pid_mode` chooses between them.

`pid_mode`	Path
0	the hand-built controller — nine blocks
1	the library block — one block

- The reason for having both is not indecision. A learner who has only used the block does not know what is inside it; a learner who has only built it by hand does not know the block exists, and will rebuild it on every project.
- **Two differences are real**, and section G measures them rather than hiding them.

Difference	Consequence
the library block differentiates its **input**, which here is the error	with $K_d = 0$ the two agree exactly; with $K_d > 0$ they do not
the library block saturates its **own** output at $[X_{\min}, X_{\max}]$	the same limits the allocation imposes, so the two saturations coincide

- The first difference has a name and a remedy. Differentiating the error puts every setpoint step through the derivative; differentiating the measurement does not. Simulink's **two-degree-of-freedom** PID block exists precisely to let the two be weighted separately.

Part 2 · Laboratory

A. Setting up (10 min)

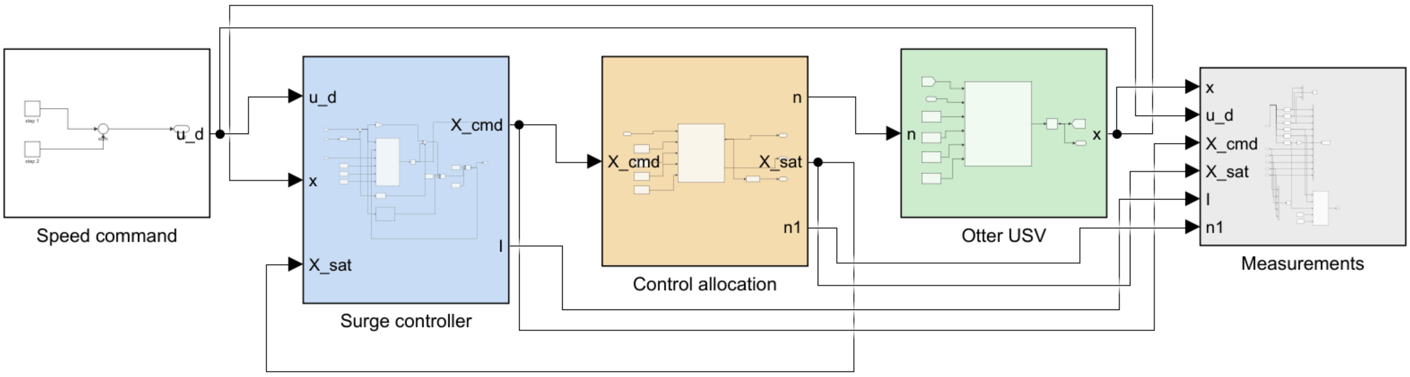
```
cd GradCourse/lectures/W02_simulink
W02_0_setup
```

Expected output:

```
  W03 setup complete
    plant           tau_u = 1.1025 s,  K_u = 0.012894 (m/s)/N
    actuator        X in [-133.42, 239.36] N  ->  u_ss in [-1.7203, 3.0864] m/s
    controller      Kp = 102, Ki = 192.38, Kd = 0, Nf = 20
    closed loop     wn = 1.5000 rad/s,  zeta = 0.7000
    anti-windup     back-calculation
    reference       1.5 -> 1.5 m/s at t = 5 s
    simulation      30 s at h = 0.02 s
```

- To restore a model that has been broken: `W02_1_build_surge_control`.

B. Reading the model (15 min)



WEEK 2 - SURGE SPEED CONTROL

The first closed loop of the course. The surge axis is a first-order,
TYPE 0 plant: u/X = K_u / (tau_u s + 1), K_u = 0.012894, tau_u = 1.1025 s.

FOUR THINGS TO TRY

1 IDENTIFY THE PLANT. loop_closed = 0, X_open = 100. Read the settled u
and the 63 per cent time. They are K_u X_open and tau_u.

2 PROPORTIONAL ONLY. Ki = 0, Kd = 0. A type 0 plant under proportional
control ALWAYS leaves a steady-state error:

u_ss / u_d = K_p K_u / (1 + K_p K_u).

Kp = 100 leaves 44 per cent. Kp = 2000 leaves 3.7 per cent and still
does not arrive. Raising the gain does not remove the error.

3 ADD THE INTEGRATOR. Ki = 192.38 with Kp = 102 places the closed-loop
poles at zeta = 0.7, wn = 1.5 rad/s. The error goes to zero exactly.
The overshoot does NOT match the textbook figure for zeta = 0.7,
because a PI controller also adds a ZERO at s = -Ki/Kp.

4 SATURATE IT. u_d1 = 3.5 m/s is not reachable: full ahead gives
exactly 3.0864 m/s. Set t_dn = 40, u_d2 = 1.5, T_final = 90 and
compare aw_mode = 0, 1, 2.

X_sat is the second feedback path, and it is the one that gets missed.
Without knowing what the propellers ACTUALLY delivered, no anti-windup
scheme can tell that the loop has been opened by the actuator.

Reading the figure

Element	Meaning
`Speed command` (white)	two steps summed, so one model covers a single step and the up-then-down profile of §F
`Surge controller` (blue)	the PID, the anti-windup selector, and the open/closed switch
`Control allocation` (sand)	$X_{\text{cmd}} \to n$, and back to X_{sat} after saturation
`Otter USV` (green)	`otter.m`, called unchanged
`Measurements` (grey)	selectors, scope, workspace log and live view

- The chain of Week 1 is now complete except for the reference model: **command → controller → allocation → plant → measurement**, in that order, left to right.
- **Two signals travel backwards, and only two.**

Signal	From	To	Why
`x`	plant	controller	the measured speed. The whole twelve-state vector is sent and the controller selects u inside
`X_sat`	allocation	controller	what the propellers **actually** delivered

> ★ `X_sat` **is the feedback path that gets missed**
>
> Without knowing what the actuator delivered, the controller has no way of telling that the demand was not followed. Every anti-windup scheme in §2-6 is built on the difference $X_{\text{cmd}} - X_{\text{sat}}$, so an allocation block that does not report what it produced makes all of them impossible.

Inside `Surge controller`

- Opening it shows the three terms, the anti-windup block that decides what reaches the integrator, and one switch:

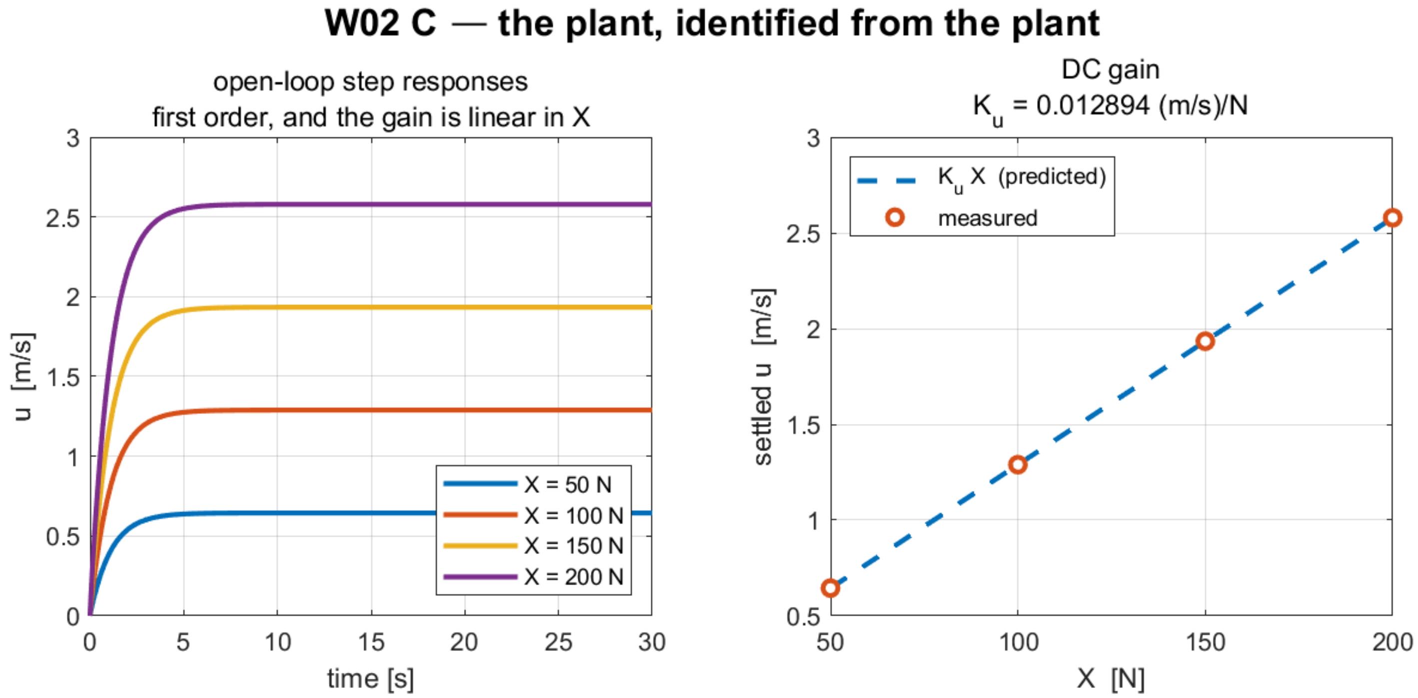
Block	Meaning
`Kp`	the proportional term
`anti-windup`	`aw_mode` selects one of the three schemes of §2-6
`I state`	the integrator, and the only state in the controller
`D filter`	$K_d N s/(s+N)$ acting on the **measurement**, not on the error
`open or closed`	`loop_closed = 0` applies `X_open` instead of the controller

- The switch is not decoration. `loop_closed = 0` turns this model into the open-loop rig of §C, so the plant that is identified is provably the plant the controller then drives.

C. Identifying the plant from the plant (15 min)

> ✎ **To produce every figure in this section**
>
> | | |
> |---|---|
> | script | `W02_C_identify_plant.m` |
> | model | `W02_surge_control.slx` |
> | figure | `img/W02_result_openloop.png` |
>
> ```
> W02_0_setup                  % once per session
> W02_C_identify_plant
> ```
>
> Opening the model and pressing **Run** produces the same figure: the scopes update as it runs and `StopFcn` draws the summary at the end.

X [N]	measured u_{ss} [m/s]	$K_u X$ [m/s]	measured τ [s]	error [%]
50	0.6447	0.6447	1.1200	-0.00
100	1.2894	1.2894	1.1200	-0.00
150	1.9341	1.9341	1.1200	-0.00
200	2.5788	2.5788	1.1200	-0.00



Reading the figure

Element	Meaning
left panel	four open-loop step responses, one per applied force
right panel, dashed	the predicted DC gain line $u = K_u X$
right panel, circles	the settled speed measured from each of the four runs

- The DC gain is exact to four decimals across the whole range. The plant really is $u = K_u X$ in steady state.
- The time constant is **not** exact: 1.1200 s measured against 1.1025 s predicted, an excess of 1.59%. The residue is the surge-pitch coupling retained by the twelve-state plant and discarded by the scalar model. It is reported rather than absorbed.

What the figure says

- **Meaning.** The loop is open — the controller is switched out and a constant force is applied directly. The left panel is the plant's response in time; the right panel collapses each of those responses to the single number that describes its steady state.
- **Trend, in numbers.** All four curves have the **same shape** and differ only in height: each rises smoothly, with no overshoot and no oscillation, and is within 2 % of its final value by about 4.5 s. Doubling the force from 100 to 200 N doubles the settled speed from 1.2894 to 2.5788 m/s exactly. The right panel is therefore a straight line through the origin, and the four measured points sit on it to four decimals.
- **Principle.** A first-order lag, $\tau_u \dot{u} + u = K_u X$, has exactly this signature: no overshoot, one time constant, and a settled value proportional to the input. **The straightness of the right panel is what says the plant is linear; the identical shape of the four curves is what says it is first order.** Both had to be checked, because the vessel underneath is a twelve-state nonlinear model and neither property was guaranteed.
- **Why identification comes before design.** Every gain in sections D to F is computed from the two numbers this figure produces, $K_u = 0.012894$ (m/s)/N and $\tau_u \approx 1.12$ s. Measuring them from the plant rather than reading them from a datasheet is what makes the later predictions checkable.
- **What changes with the situation.** Nothing here depends on a controller, because there is none. Adding force moves the vessel up the same line; it never changes the line. The line does end, though — at $X = 239.36$ N the propellers saturate, and the speed ceiling that follows is the one number no controller in this week can argue with.

★ $u_{\max} = U_{\max}$ **is exact, and it is not a coincidence**

$X_{\max} = 2k_{\text{pos}}n_{\max}^2 = 24.4g$ by construction of $n_{\max}$, and $X_u = -24.4g/U_{\max}$ by construction of the damping. The two $24.4g$ cancel, leaving $u_{\max} = U_{\max} = 3.0864$ m/s exactly. No controller of any structure can ask for more.

D. Proportional only (15 min)

✎ **To produce every figure in this section**

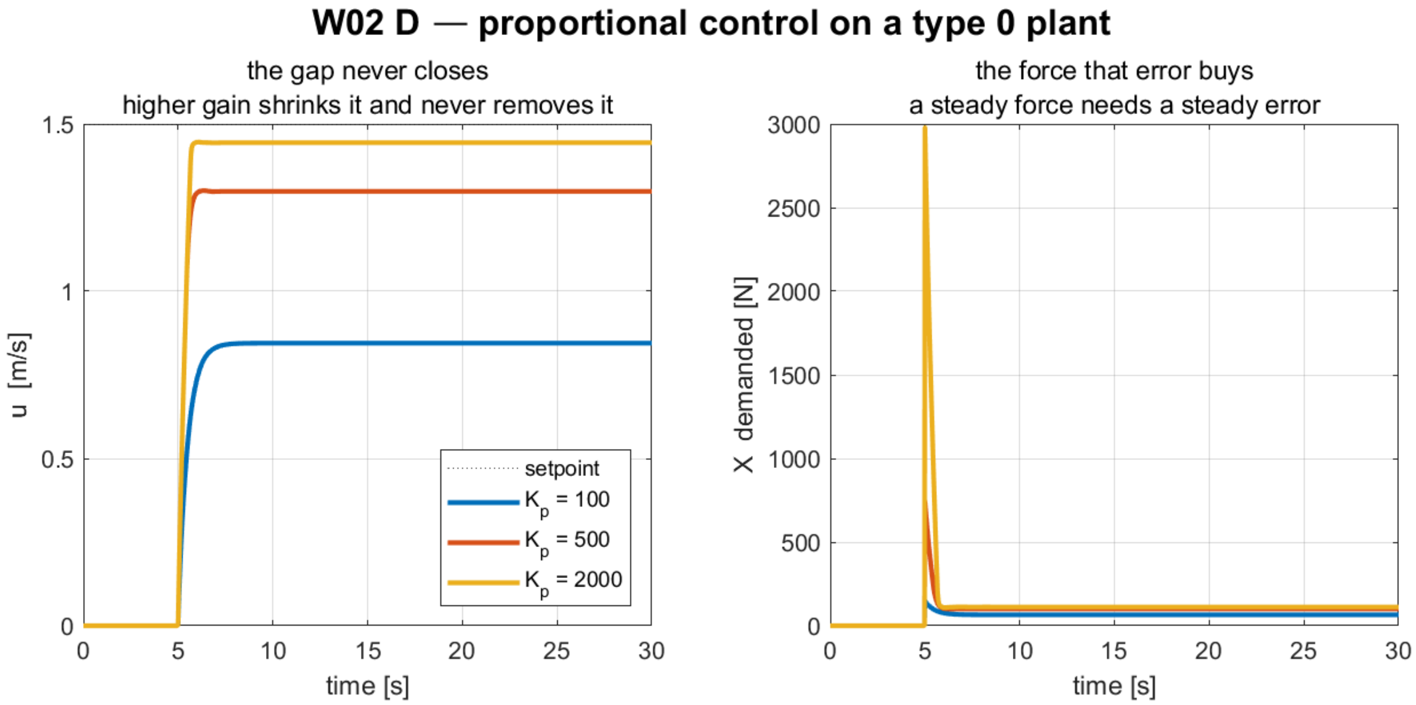
script	`W02_D_proportional_only.m`
model	`W02_surge_control.slx`
figure	`img/W02_result_P.png`

```
W02_0_setup                    % once per session
W02_D_proportional_only
```

Opening the model and pressing **Run** produces the same figure: the scopes update as it runs and `StopFcn` draws the summary at the end.

Measured with $u_d = 1.5$ m/s, $K_i = K_d = 0$:

K_p	K_pK_u	predicted u_{ss}	measured u_{ss}	error [%]	X_{ss} [N]
100	1.2894	0.8448	0.8448	43.68	65.519
500	6.4471	1.2986	1.2986	13.43	100.711
2000	25.7883	1.4440	1.4440	3.73	111.989



Reading the figure

Element	Meaning
left panel	three step responses; none reaches the dashed setpoint
right panel	the force each controller demanded, transient and steady

- Prediction and measurement agree to four decimal places, because both are consequences of the same structural fact and neither involves an approximation.
- A twentyfold increase in gain buys a reduction from 44% to 3.7%. The right panel shows why the error cannot vanish: the steady force needed is 111.99 N at the highest gain, and a proportional controller can only produce it from a non-zero error.

What the figure says

- **Meaning.** One setpoint, 1.5 m/s, and three proportional controllers. The left panel is how close each got; the right panel is what each had to demand to get there.
- **Trend, in numbers.** Raising K_p from 100 to 2000 moves the settled speed from 0.8448 to 1.4440 m/s — closer at every step, and **short at every step**. The remaining gap shrinks from 44% to 13% to 3.7%: each twentyfold increase in gain buys roughly one decimal place, and none of them buys the last one. Meanwhile the peak demand explodes from about 700 to 3000 N, more than ten times what the propellers can deliver.
- **Principle.** The plant has no free integrator, so the loop is **type 0** and the final-value theorem gives $u_{ss}/u_d = K_pK_u/(1 + K_pK_u)$ — a ratio that approaches one and never reaches it. Holding a speed requires a permanent force to balance the drag; a proportional controller manufactures force only from error; **so the error is what pays for the force, and it cannot be zero.**
- **Why the right panel settles rather than falling to zero.** All three demands converge to roughly 65–112 N and stay there. That plateau *is* the drag at the speed each vessel reached. Compare Week 3, where the same panel returns to zero because a vessel that has stopped turning needs no moment: the structural difference between the two axes is visible in this one plot.
- **What changes with the situation.** Raising the gain here costs actuator authority to buy accuracy, and both run out: at $K_p = 2000$ the transient demand is an order of magnitude past saturation, so the response the figure shows is already partly fictional. Section E adds the integral term, which supplies the steady force from a **zero** error and settles the question rather than shrinking it.

E. Integral and derivative (20 min)

✎ To produce every figure in this section

script	`W02_E_integral_and_derivative.m`
model	`W02_surge_control.slx`
figure	`img/W02_result_PI.png and img/W02_result_D.png`

```
W02_0_setup                    % once per session
W02_E_integral_and_derivative
```

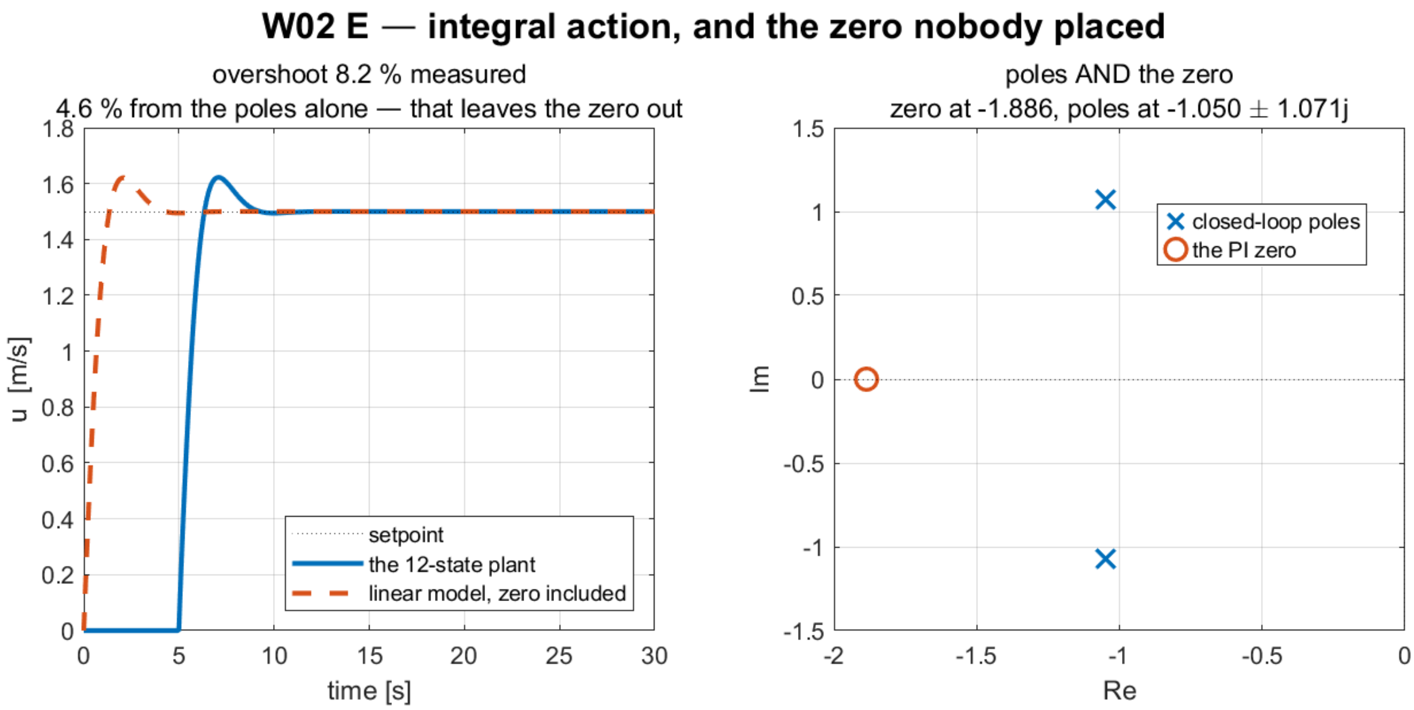
Opening the model and pressing **Run** produces the same figure: the scopes update as it runs and `StopFcn` draws the summary at the end.

PI

Designed for $\zeta = 0.7$, $\omega_n = 1.5$ rad/s, giving $K_p = 102.00$ and $K_i = 192.38$, with the zero at $s = -1.8861$ and the poles at $-1.050 \pm 1.071\mathrm{j}$.

Source	overshoot [%]	t_s (2%) [s]
poles only, $\exp\left(-\pi\zeta/\sqrt{1-\zeta^2}\right)$	4.60	3.81
linear model **including** the PI zero	8.07	3.47
the twelve-state plant	8.15	3.46

- Steady-state error at $t = 30$ s: -2.1×10^{-12} m/s, that is, zero to the tolerance of the solver.



Reading the figure

Element	Meaning
left panel, solid	the twelve-state plant
left panel, dashed	the linear model **with the PI zero included**
right panel, crosses	the two closed-loop poles, placed by the design
right panel, circle	the PI zero at $s = -K_i/K_p$, which the design never mentioned

- The second and third rows of the table agree to 1%. The first row disagrees with both by 75%. The zero, not the plant, is the source of the discrepancy.

What the figure says

- **Meaning.** The left panel compares the real plant against the linear model it was designed with. The right panel shows what that model actually contains — and the circle is the part the design procedure never asked about.
- **Trend, in numbers.** Both curves in the left panel rise, overshoot to about 1.62 m/s, and settle on 1.5 m/s with no residual error at all: the integral term closed the gap that section D could not. The measured overshoot is 8.15% and the linear model gives 8.07% — agreement to a tenth of a point. **The textbook figure for $\zeta = 0.7$ is 4.6%, and it is wrong here by a factor of nearly two.**
- **Principle.** ζ and ω_n describe the **poles**. A PI controller does not only place poles; it also places a zero at $s = -K_i/K_p = -1.886$, and a zero close to the poles raises overshoot. The right panel makes the proximity visible: the zero sits at -1.886 while the poles sit at $-1.050 \pm 1.071j$, barely further out than the poles themselves.
- **Why the two left-hand curves agree and the textbook does not.** The dashed model includes the zero, so it matches the plant. The 4.6% figure comes from a formula derived for a system with **no zero at all**. The damping

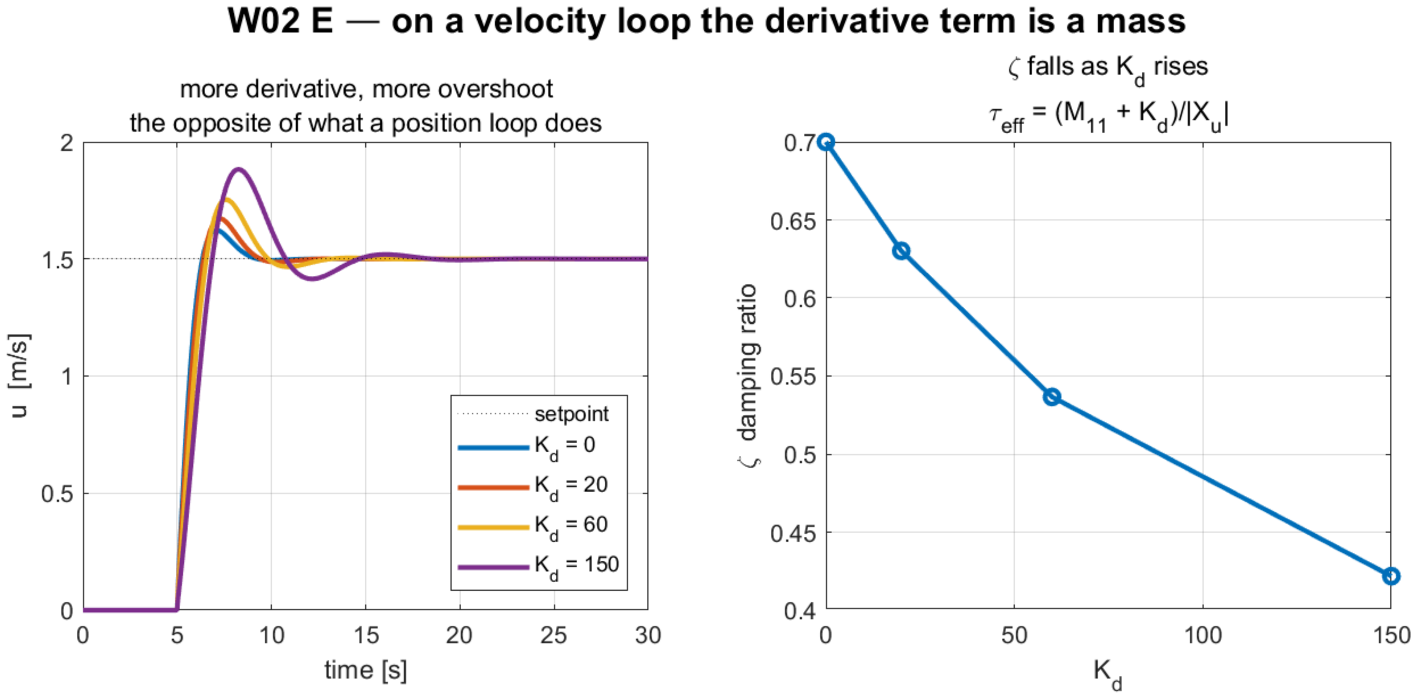
ratio was designed correctly; the prediction made from it was not. Diagnosing the discrepancy as a plant modelling error would have been the wrong conclusion, and the pole-zero map is what rules it out.

- **What changes with the situation.** Moving the zero further left — a smaller K_i/K_p — makes the textbook figure accurate again, at the cost of a slower recovery from a load change. Section G shows a second way of shifting responsibility for the derivative term, and Week 8's reference model is the general remedy: shape the setpoint instead of arguing with the closed-loop zeros.

Derivative

K_p and K_i held at the values above:

K_d	$M_{11} + K_d$ [kg]	ω_n	ζ	predicted M_p [%]	measured M_p [%]
0	85.50	1.5000	0.7000	8.07	8.15
20	105.50	1.3504	0.6302	11.60	11.48
60	145.50	1.1499	0.5366	17.39	16.88
150	235.50	0.9038	0.4218	26.56	25.50



Reading the figure

Element	Meaning
left panel	four step responses; overshoot grows monotonically with K_d
right panel	the damping ratio ζ computed from the effective mass $M_{11} + K_d$, falling as K_d rises

- $K_d = 150$ N per m/s² makes an 85.5 kg vessel behave like a 235.5 kg one, and ζ falls from 0.700 to 0.422. Prediction and measurement agree to within 1.1 percentage points across the whole sweep.
- The correct conclusion is not that derivative action is useless. It is that a term must be substituted into the equation of motion before its effect is assumed.

What the figure says

- **Meaning.** K_p and K_i are held at the values designed in the previous figure, and only K_d changes. The left panel is the consequence; the right panel is the reason.
- **Trend, in numbers.** Overshoot **rises** with derivative gain — 8.15%, 11.48%, 16.88%, 25.50% — and the $K_d = 150$ response develops a visible second swing that the $K_d = 0$ response does not have. The right panel

falls in step, ζ dropping from 0.700 to 0.422. The two panels are the same statement told twice.

- **Principle.** Substituting the control law into the surge equation gives $(M_{11} + K_d)\dot{u} + |X_u|u = \ldots$ — the derivative gain lands **beside the mass**, not beside the damping. Adding $K_d = 150$ therefore does not damp the vessel; it makes it **heavier**, from 85.5 to 235.5 kg, and a heavier vessel with the same damping is less damped.
- **How this differs from Week 3.** The identical term, in an identical PID controller, reduces overshoot on the heading axis and increases it here. The difference is not the controller and not the tuning: on a **velocity** loop the derivative of the controlled variable is an acceleration, and acceleration multiplies mass. On an **angle** loop it is a rate, and rate multiplies damping. **The term did not change; the axis did.**
- **What changes with the situation.** Nothing about this can be tuned away, so the practical answer on a speed loop is $K_d = 0$, which is what the rest of this week uses. Section I revisits the derivative term on a plant where it is genuinely wanted, and shows what taking it honestly costs.

F. Windup (25 min)

✎ To produce every figure in this section

script	`W02_F_windup.m`
model	`W02_surge_control.slx`
figure	`img/W02_result_windup.png`

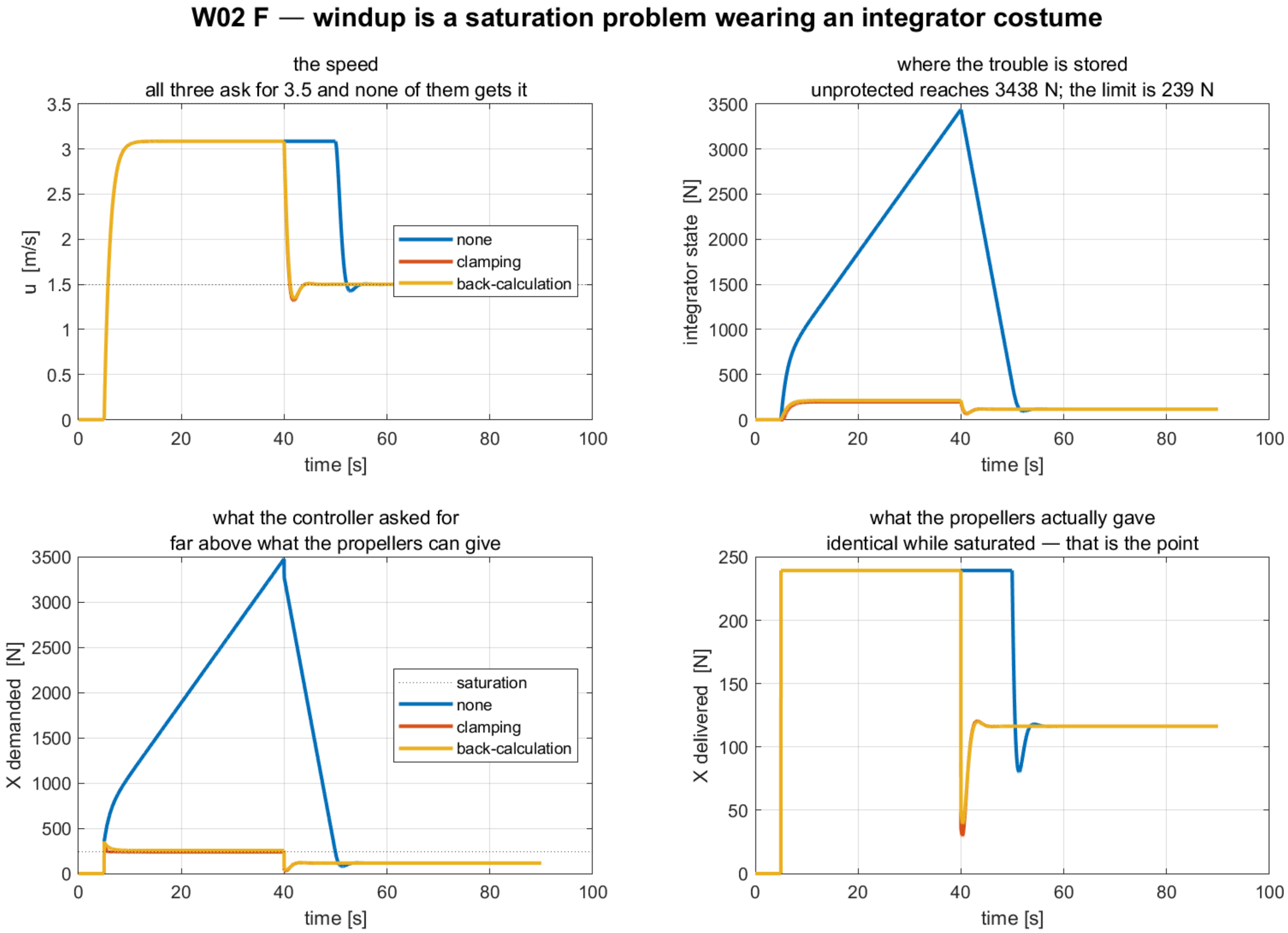
```
W02_0_setup                    % once per session
W02_F_windup
```

Opening the model and pressing **Run** produces the same figure: the scopes update as it runs and `StopFcn` draws the summary at the end.

- $u_d = 3.5$ m/s is applied at $t = 5$ s. It is **unreachable**: $u_{\max} = 3.0864$ m/s. At $t = 40$ s the demand drops to 1.5 m/s, which is reachable.

Anti-windup	peak I [N]	peak X_{cmd} [N]	recovery [s]
none	3438.2	3480.0	13.980
clamping	197.8	355.9	3.400
back-calculation	339.0	464.1	3.640

- **Recovery** is measured from $t = 40$ s to the last instant at which $|u - 1.5| > 0.03$ m/s, that is, a 2% band on the new setpoint. The averaging window is the whole interval $[40,\ 90]$ s.



Reading the figure

Element	Meaning
top left	the speed; without anti-windup the vessel holds 3.09 m/s for ten seconds after the demand has dropped
top right	the integrator state, reaching 3438 N against a largest useful value of 239 N
bottom left	the force the controller **asked for**, with the saturation limit dotted
bottom right	the force the propellers **delivered** — identical in all three runs while saturated

- The integrator accumulates 14.4 times the largest force the actuator can deliver. Every newton of that has to be integrated back out before the loop responds at all.
- Clamping and back-calculation differ little here — 3.40 s against 3.44 s. Clamping is slightly faster because it stops the integrator completely; back-calculation is smoother at the boundary and has one gain to choose. On this plant the choice is not important, which is itself worth knowing.

What the figure says

- **Meaning.** Four views of the same three runs. The two left panels are what happened and what was asked for; the two right panels are where the trouble was stored and what the water actually received.
- **Trend, in numbers.** For the first 40 s **all three runs are identical** in the top-left and bottom-right panels: every controller sits on the limit, delivers 239 N, and drives the vessel to 3.09 m/s. Nothing distinguishes them. They separate the instant the setpoint drops to a reachable 1.5 m/s: the protected runs recover in 3.4 s while the unprotected one takes 13.98 s, holding 3.09 m/s for ten seconds after being told to slow down.
- **Principle.** While the demand is impossible the error never changes sign, so the integrator keeps accumulating — to 3438 N, a force the propellers can never produce. The top-right panel is the whole diagnosis: **the difference between the three runs lives entirely in the integrator, and nowhere else.**

- **Why the bottom-right panel matters more than the bottom-left.** The bottom-left panel shows demands that differ by a factor of fourteen. The bottom-right shows that the water felt **exactly the same force** in all three. A demand above saturation is not a control action; it is a number in a register. The loop was open, and no gain chosen inside a closed-loop analysis can account for a loop that is not closed.
- **What separates the two remedies.** Clamping stops the integrator where it happens to be; back-calculation steers it towards the value that makes the demand equal the limit. They settle at different integrator values, which is why raising the back-calculation gain never turns one into the other — section H measures that on a plant simple enough to see it.

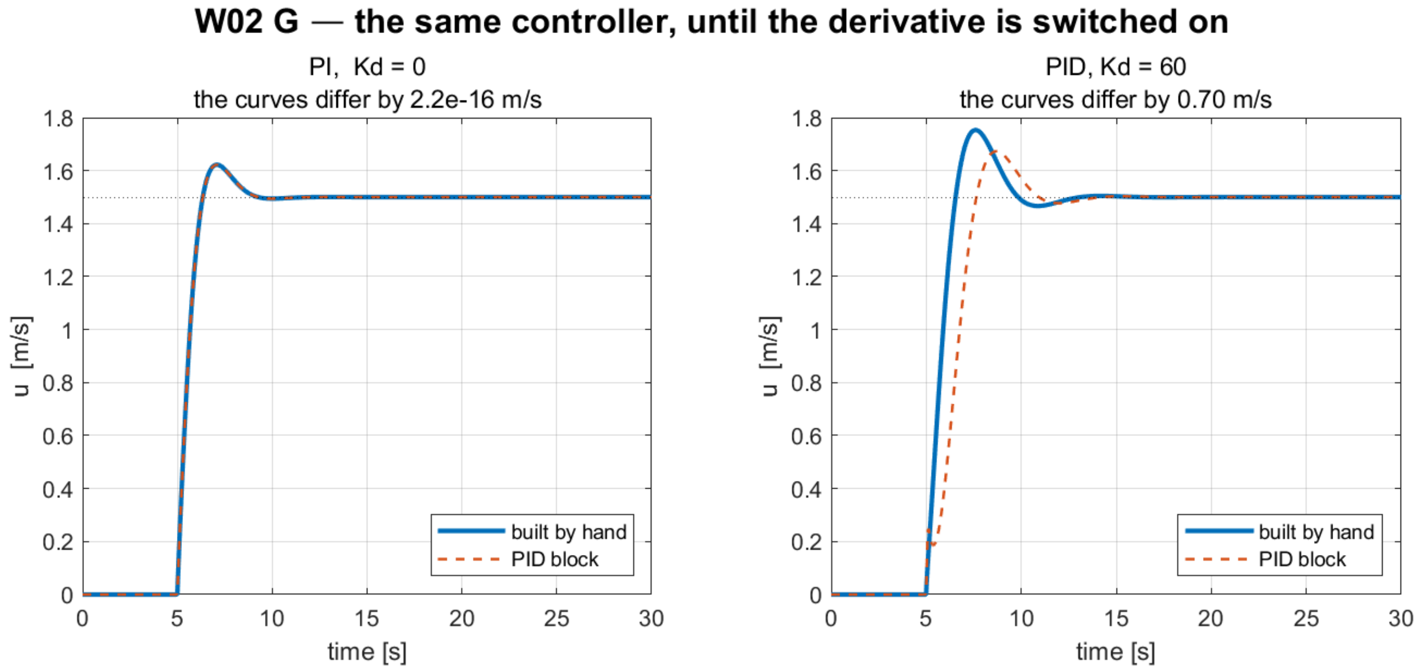
> **★ Nothing was wrong with the integrator**
>
> It did exactly what an integrator does. The loop had been opened by the actuator, and no gain chosen inside a closed-loop analysis can account for a loop that is not closed. Every one of the four experiments this week was predicted correctly by the linear model **except** where saturation intervened, and that is the boundary of linear design.

G. Hand-built against the library block (10 min)

> **✎ To produce every figure in this section**
>
> | | |
> |---|---|
> | script | `W02_G_block_vs_handbuilt.m` |
> | model | `W02_surge_control.slx` |
> | figure | `img/W02_result_block.png` |
>
> ```
> W02_0_setup                   % once per session
> W02_G_block_vs_handbuilt
> ```
>
> Opening the model and pressing **Run** produces the same figure: the scopes update as it runs and `StopFcn` draws the summary at the end.

The same two runs, with `pid_mode = 0` and `pid_mode = 1`:

Case	largest gap in u [m/s]	M_p hand-built [%]	M_p library block [%]
PI, $K_d = 0$	2.2×10^{-16}	8.15	8.15
PID, $K_d = 60$	1.8×10^{-1}	16.88	11.40



Reading the figure

Element	Meaning
left panel	$K_d = 0$: the thick and dashed traces are one curve
right panel	$K_d = 60$: they separate, and the library block overshoots less

- With $K_d = 0$ the two agree to **machine precision**. Nine blocks and one library block compute the same thing, and the back-calculation written by hand in §2-6 is the back-calculation the block implements.
- With $K_d = 60$ they differ by 0.70 m/s at the peak. Neither is wrong. The library block differentiates the **error**, so a step in u_d passes through its derivative and produces a kick that the hand-built path — which differentiates the **measurement** — never sees.

What the figure says

- **Meaning.** The same plant, the same gains and the same setpoint, controlled twice: once by nine blocks assembled from the equations of §2-4, once by Simulink's PID Controller block. The two panels differ only in whether the derivative term is switched on.
- **Trend, in numbers.** In the left panel the two traces are **one curve**: the largest difference anywhere in the run is 2.2×10^{-16} m/s, which is machine precision. In the right panel they part company during the transient — the hand-built path peaks at 1.76 m/s and the block at 1.68, overshoots of 16.88% against 11.54% — and then converge again, ending on the same 1.5 m/s.
- **Principle.** With $K_d = 0$ the two implementations are algebraically identical, and the left panel is the proof. That agreement is what licenses using the library block for the rest of the course: it is not a different algorithm, only a shorter way of writing the same one.
- **Where the difference comes from.** The library block differentiates its **input**, which is the error. The hand-built path differentiates the **measurement**. A step in u_d therefore passes straight through the block's derivative and produces a kick; a measurement never steps, so the hand-built path sees nothing. **Both are correct implementations of a PID controller, and the phrase "a PID controller" is not specific enough to distinguish them.**
- **What changes with the situation.** The gap appears only at a setpoint change and vanishes in steady state, so a loop that mostly rejects disturbances will never notice, and a loop that mostly follows commands will notice a great deal. Simulink's two-degree-of-freedom PID block exists to weight the two paths separately, which is the general answer to the question this figure raises.

✎ **Which to use**

The library block, in almost every case. It is one block, it is tested, and it offers discrete-time forms, external reset and tracking mode that would each take several more blocks by hand. Build it by hand once, to know what is inside it, and then stop.

H. The principle on its own (20 min)

✎ **To produce every figure in this section**

script	`W02_H_antiwindup_run.m`
model	`W02_H_antiwindup.slx`
figure	`img/W02_result_aw_principle.png and img/W02_result_aw_Kb.png`

```
W02_0_setup                  % once per session
W02_H_antiwindup_run
```

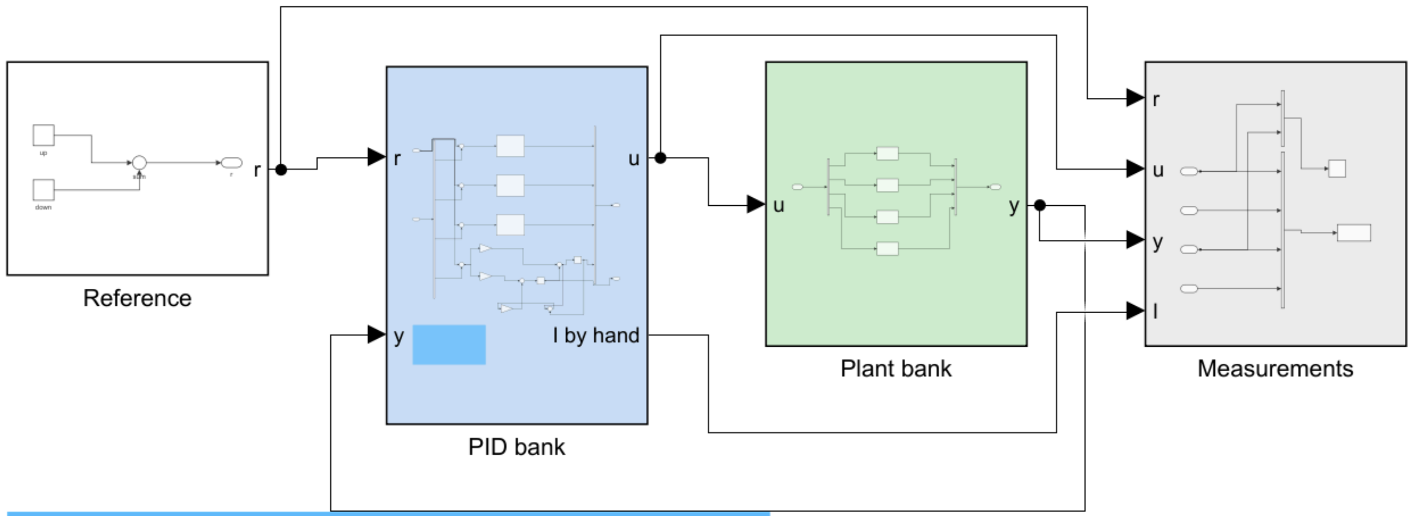
Opening the model and pressing **Run** produces the same figure: the scopes update as it runs and `StopFcn` draws the summary at the end.

- The vessel is a distraction from the mechanism. A second model strips everything away:

$$G(s) = \frac{1}{s+1}, \qquad u \in [-1,\ +1], \qquad \text{PI with } K_p = 1.8,\ K_i = 4.$$

- The DC gain is 1 and $|u| \le 1$, so the largest reachable output is $y = 1$. The reference is stepped to 2 — unreachable — held for 14 s, then dropped to 0.5, which is reachable.

```
W02_H_build_antiwindup
W02_H_antiwindup_run
```



WEEK 2, THE ANTI-WINDUP PRINCIPLE ON ITS OWN

No vessel, no propellers, no twelve states. A first-order plant, a limit, and an integrator that does not know about the limit:

G(s) = 1 / (s + 1),   u in [-1, +1],   PI control.

The DC gain is 1 and |u| <= 1, so the largest reachable output is y = 1. The reference is stepped to 2, which CANNOT be reached, held, and then dropped to 0.5, which can.

FOUR CONTROLLERS, IDENTICAL GAINS, ONE PLANT EACH

1 Simulink PID block, anti-windup = none
2 Simulink PID block, anti-windup = clamping
3 Simulink PID block, anti-windup = back-calculation, gain Kb
4 the same back-calculation built by hand

Rows 3 and 4 must agree to machine precision. That agreement is what licenses the claim that the library block does what the theory says.

WHAT TO WATCH

Not the rise to y = 1 - all four are identical there. Watch what happens AFTER t = 15 s, when the reference becomes reachable again. Row 1 sits at y = 1 for several seconds doing nothing, because the integrator has to unwind a charge it should never have accumulated.

Then open PID bank and read the annotation on row 4.

Reading the figure

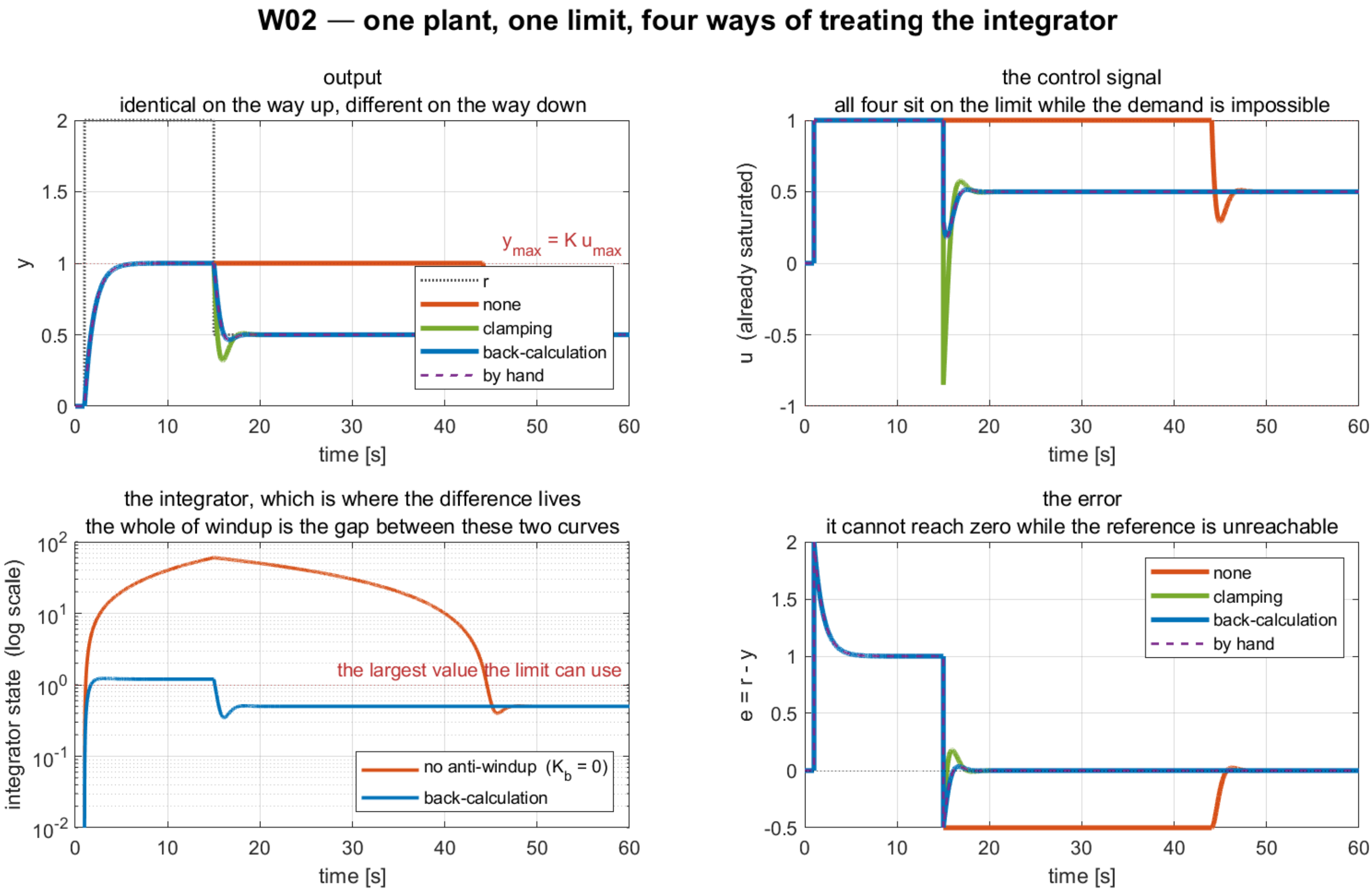
Element	Meaning
`Reference` (white)	the step up to an impossible value, then down to a possible one
`PID bank` (blue)	four controllers with identical gains: three library blocks with the three anti-windup settings, and the fourth built by hand
`Plant bank` (green)	four identical copies of $1/(s+1)$, one per controller
`Measurements` (grey)	the log

- The fourth row exists so that the **integrator state is a signal**. The library block does not expose it, and the integrator is where the whole phenomenon lives.

Results

Anti-windup	time on the limit [s]	recovery [s]	y at $t = 20$ s
none	43.05	32.04	1.0000
clamping	14.00	2.380	0.4997
back-calculation	14.00	2.645	0.5003
the same, by hand	14.00	2.645	0.5003

- **Recovery** is the time after $t = 15$ s for y to enter and stay within a 2% band on 0.5.



Reading the figure

Element	Meaning
top left	the output; all four are one curve on the way up
top right	the control signal; all four sit on the limit for the same 14 s
bottom left	the integrator state on a log scale, with and without anti-windup
bottom right	the error, which cannot reach zero while the reference is unreachable

- **All four are identical on the way up.** While the demand is impossible every scheme sits on the limit and produces $y = 1$. Nothing distinguishes them until $t = 15$ s.
- The bottom-left panel is the whole phenomenon. Without anti-windup the integrator winds to 60.00, against a largest useful value of 1.0 and against 1.225 with back-calculation — a factor of 49. It then takes 30 s to unwind, and the loop does nothing for all of it.
- Rows 3 and 4 agree to **exactly zero**. The library block and the hand-built path are the same algorithm, so the theory of §2-6 is a correct description of what the block does.

What the figure says

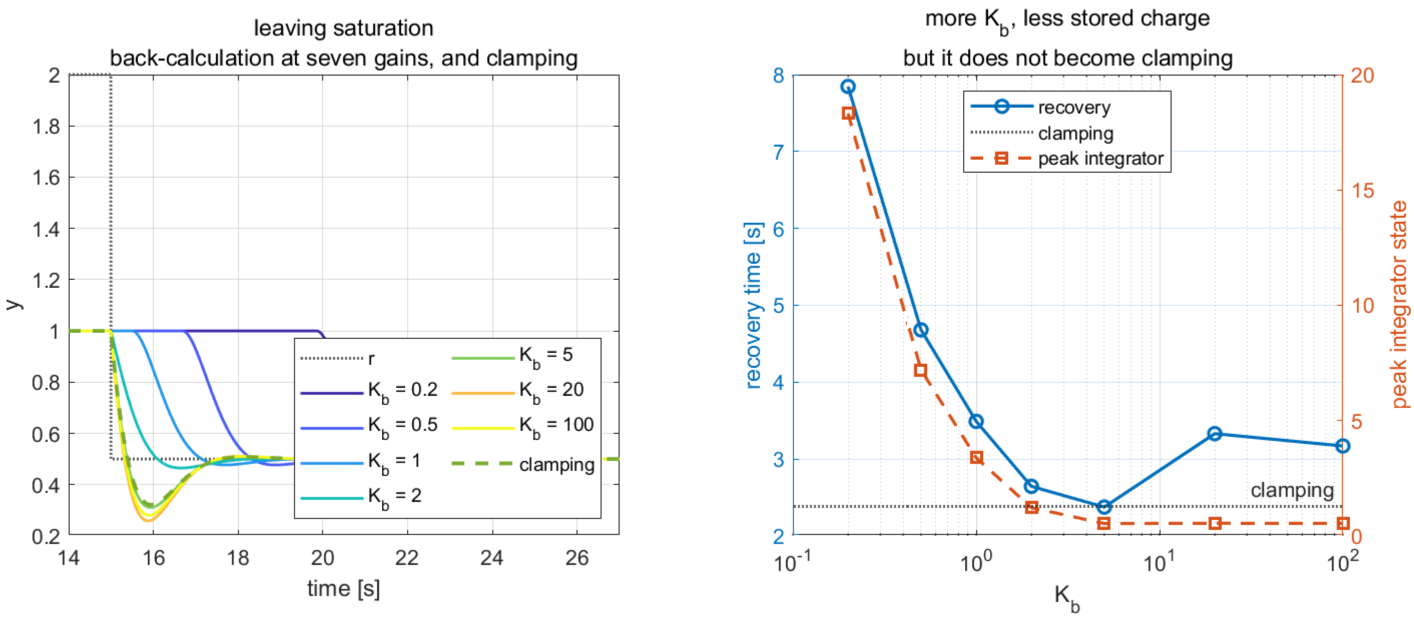
- **Meaning.** The plant is $1/(s+1)$ with the actuator limited to ±1, chosen so that nothing in the picture belongs to a boat. Four integrator treatments, four panels: what the plant did, what the actuator received, what the integrator held, and what the error was doing all along.
- **Trend, in numbers.** For the first 14 s **all four curves coincide in three of the four panels**. The output reaches $y = 1$ and stops there, because $y_{\max} = Ku_{\max} = 1$ and the reference asked for 2. The control signal sits flat on the limit. The error sits flat at $+1$. Only the bottom-left panel separates them, and it does so by a factor of 49: the unprotected integrator climbs to 60.00 while back-calculation holds 1.225.
- **Principle.** An integrator integrates. While the reference is unreachable the error cannot change sign, so the integral grows without bound — and it grows into a quantity the actuator cannot use, since the largest demand the limit can accept corresponds to an integrator value of 1.0. **The trouble is not that the integrator misbehaved; it is that the loop was open and the integrator was not told.**
- **Why the bottom-left panel is drawn on a log scale.** Linearly, the back-calculation curve would be a flat line against the axis and the reader would see one curve and a wall. The log scale shows both, and shows that the protected integrator is not merely smaller but **stays in the range the actuator can act on** for the whole run.
- **What separates the four schemes, and when.** Nothing, until $t = 15$ s. When the reference becomes reachable the unprotected loop must first unwind 30 s of stored demand and takes 32.04 s to settle; the others take about 2.5 s. **A scheme that is invisible for 14 s and decisive at second 15 cannot be evaluated on the response to a reachable setpoint** — which is why this experiment commands an impossible one.

What the back-calculation gain does

K_{aw}	recovery [s]	peak integrator	undershoot after [%]
0.2	7.845	18.308	4.60
0.5	4.680	7.199	4.60
1	3.490	3.389	4.60
2	2.645	1.225	7.02
5	2.375	0.538	37.84
20	3.330	0.548	48.25
100	3.170	0.544	43.96

- clamping, for comparison: recovery 2.380 s

W02 — back-calculation and clamping are different schemes



Reading the figure

Element	Meaning
left panel	the moment of leaving saturation, at seven values of K_{aw}, with clamping dashed
right panel, left axis	recovery time, with a minimum near $K_{\mathrm{aw}} = 5$
right panel, right axis	the peak integrator state, falling and then flattening

- There is a **minimum**, near $K_{\mathrm{aw}} = 5$. Below it the integrator is not pulled back fast enough; above it the loop leaves saturation abruptly and undershoots, from 4.60% to 43.96%.
- Raising K_{aw} does **not** turn back-calculation into clamping. The two remain different at every gain, and the reason is §2-6: clamping stops the integrator wherever it happens to be, while back-calculation steers it to the value that makes the demand equal the limit. They are different fixed points, not two ends of one scale.

What the figure says

- **Meaning.** The left panel zooms in on the one moment that matters — the twelve seconds around leaving saturation — for seven back-calculation gains and, dashed, for clamping. The right panel reduces each of those runs to two numbers.
- **Trend, in numbers.** The two curves in the right panel do **not** move together. The peak integrator state falls steeply and then flattens, from 18.3 at $K_{\mathrm{aw}} = 0.2$ to about 0.54 at $K_{\mathrm{aw}} = 5$ and no lower afterwards. Recovery time falls too, from 7.845 to 2.375 s, and then **turns and rises again** to 3.33 s. In the left panel the reason is visible: the high-gain runs dive to $y \approx 0.27$ before recovering, an undershoot that grows from 4.60% to 43.96%.
- **Principle.** Back-calculation feeds the excess demand back into the integrator through K_{aw}. A larger gain empties the integrator faster, which shortens the recovery — until the emptying is so abrupt that the loop leaves the limit with a jolt and has to recover from that instead. **The minimum near $K_{\mathrm{aw}} = 5$ is where those two effects cross.**
- **Why the dotted clamping line never becomes an asymptote.** If clamping were the $K_{\mathrm{aw}} \to \infty$ limit of back-calculation, the blue curve would approach the dotted line from above and stay there. It does not: it dips **below** clamping near $K_{\mathrm{aw}} = 5$ and comes back up. The two schemes drive the integrator to different values, so no gain makes one into the other.
- **What changes with the situation.** The rule of thumb $K_{\mathrm{aw}} = 1/\tau$ puts this plant at $K_{\mathrm{aw}} = 1$, giving 3.49 s against a best available 2.375 s — a reasonable default and not an optimum. The sweep costs one line to run, and on a plant where saturation is routine it is worth running rather than guessing.

★ The rule of thumb, and its limit

$K_{\text{aw}} = 1/\tau$ is the usual starting point, and on this plant $\tau = 1$ s gives $K_{\text{aw}} = 1$ — a recovery of 3.49 s against the best available 2.375 s. It is a reasonable default and it is not optimal. The sweep above takes one line to run, and the result is a genuine trade-off between how fast the integrator is emptied and how violently the loop leaves the limit.

I. The pseudo-derivative, on a plant that wants one (25 min)

✎ To produce every figure in this section

script	`W02_I_pseudo_derivative_run.m`
model	`W02_I_pseudo_derivative.slx`
figure	`img/W02_result_pd.png and img/W02_result_pd_N.png`

```
W02_0_setup                    % once per session
W02_I_pseudo_derivative_run
```

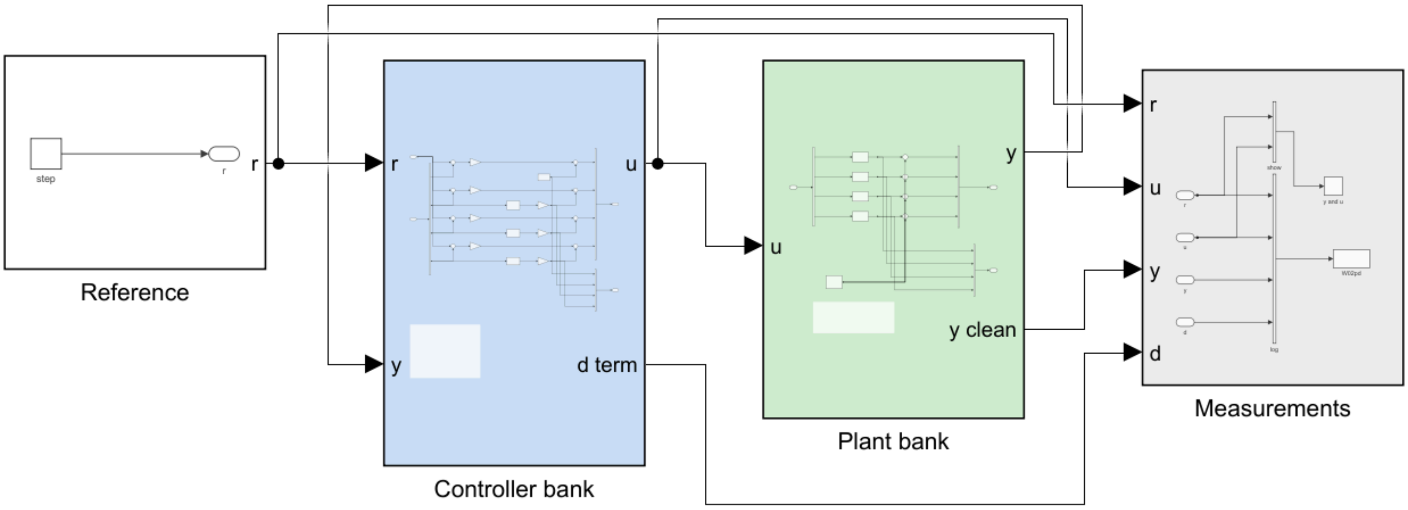
Opening the model and pressing **Run** produces the same figure: the scopes update as it runs and `StopFcn` draws the summary at the end.

- §2-4 found that on the surge axis the derivative term acts as a **mass** and makes things worse, so nothing in sections A to G shows what the derivative filter is *for*.
- A third model uses a plant where derivative action is genuinely wanted:

$$G(s) = \frac{1}{s^2 + 0.4\,s}, \qquad K_p = 4,\ K_d = 2, \qquad \text{measurement noise } \sigma = 0.01.$$

- Under proportional control alone the closed loop is $s^2 + 0.4s + K_p$, so $\zeta = 0.2/\sqrt{K_p} = 0.1$. It rings badly. Derivative action is the cure, which makes *how to take the derivative* a real question.

```
W02_I_build_pseudo_derivative
W02_I_pseudo_derivative_run
```



WEEK 2, SECTION I - THE PSEUDO-DERIVATIVE

A PID controller never contains a differentiator. This model shows why, on a plant where derivative action is actually wanted.

G(s) = 1 / (s^2 + 0.4 s)   zeta = 0.1 under P control alone

FOUR ROWS, SAME GAINS, SAME NOISE

1 P only   rings, but the actuator is quiet
2 Kd s   ideal   damps well, actuator unusable
3 Kd N1 s/(s+N1) pseudo   N1 large - still noisy
4 Kd N2 s/(s+N2) pseudo   N2 small - quiet, slightly slower

THE MECHANISM

Differentiation has gain |w|: it multiplies every component of the signal by its own frequency. Noise is the fastest thing in the measurement, so differentiation finds it and amplifies it more than anything else. The pseudo-derivative is the same operator with its gain capped at N, so N is the knob that decides how much of the noise reaches the actuator.

The cost is phase lag below N. Choosing N is therefore a trade, and the runner measures both sides of it.

Edit pd_Kp, pd_Kd, pd_N1, pd_N2 and pd_noise in W02_0_setup.m.

Reading the figure

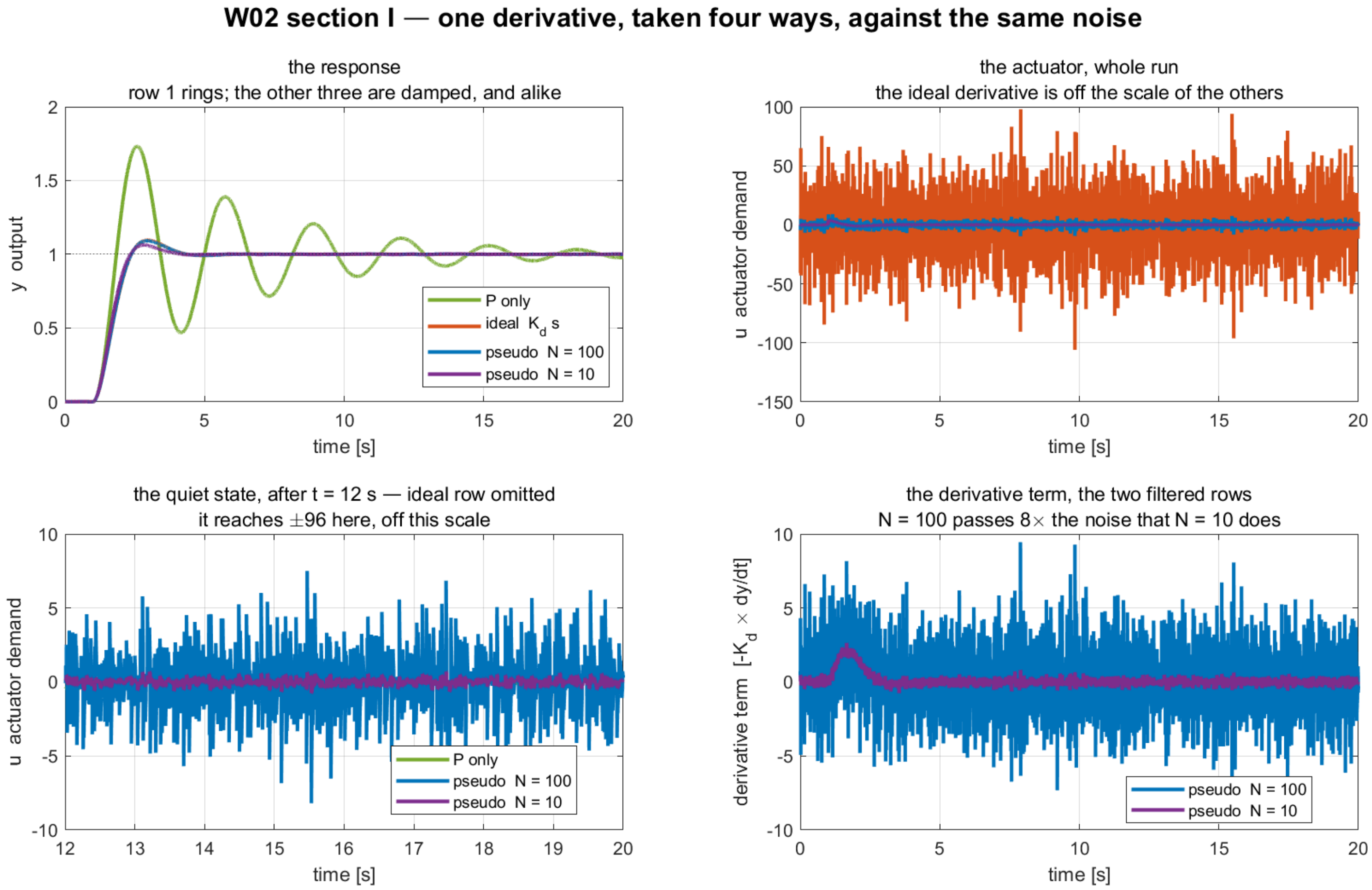
Element	Meaning
`Reference` (white)	one step, at $t = 1$ s
`Controller bank` (blue)	four controllers, identical except for the block in the derivative slot: none, ideal s, and the pseudo-derivative at two values of N
`Plant bank` (green)	four copies of $G(s)$ and **one** noise source shared by all four
`Measurements` (grey)	the log, which records the clean output so the plots are not themselves noisy

> **★ One noise source, not four**
>
> Four independent noise generators would make the four control signals differ for two reasons at once, and the comparison would prove nothing. Sharing one realisation is what licenses the claim that the differences below are caused by the derivative implementation alone.

① The four rows, with noise

Derivative	overshoot	settling	RMS(u) quiet	max $\|u\|$
P only	73.22%	19.00 s	0.1814	4.04
ideal, $K_d s$	9.54%	2.99 s	8.7452	105.94
pseudo, $N = 100$	9.10%	2.97 s	1.6780	9.61
pseudo, $N = 10$	6.28%	2.63 s	0.2348	4.26

- **RMS(u) quiet** is the standard deviation of the actuator demand after $t = 12$ s, when the setpoint has been reached and everything remaining in the signal is noise.



Reading the figure

Element	Meaning
top left	the outputs. Row 1 rings for the whole run; the other three are damped and nearly identical
top right	the actuator demand. The ideal derivative reaches ± 106 on a plant whose steady demand is 4
bottom left	the quiet state, **with the ideal row omitted** so the other three are visible at all
bottom right	the derivative term alone, for the two filtered rows

- The ideal derivative does its control job perfectly and destroys the actuator doing it. Its peak demand is 106 — **twenty-six times** the steady demand — and all of it is noise.

What the figure says

- **Meaning.** Four controllers, one plant, one shared realisation of the measurement noise. The left column is the useful signal; the right column is what the actuator was asked to do while producing it.

- **Trend, in numbers.** The top-left panel says the derivative is genuinely wanted here: row 1 rings for the whole 20 s and overshoots 73.2%, while the other three settle in about 3 s at under 10%. **On that panel the three derivative rows are almost indistinguishable.** The top-right panel says they are nothing alike: the ideal derivative swings ± 106 against a steady demand of 4, and quiet-state RMS runs 0.18, 8.75, 1.68, 0.23 across the four rows — a factor of 38 between the ideal and the slower filter.
- **Principle.** Differentiation has gain $|j\omega|$, which grows without bound. It therefore finds the fastest thing in the measurement — the noise — and multiplies it by the largest number available. The pseudo-derivative $Ns/(s+N)$ has the same gain below N and levels off at N above it, so **it is the same operator with a ceiling on how much it may amplify.**
- **Why the bottom-left panel omits the ideal row.** With it included, the other three collapse onto the axis and nothing can be read. Leaving it out is not hiding the result; the number ± 96 is printed on the panel, and the omission is what makes the remaining comparison visible at all.
- **What separates $N = 100$ from $N = 10$.** The bottom-right panel isolates the derivative term itself: the fast filter passes eight times the noise the slow one does, for a response that is 2.8 points *worse* in overshoot. Above a certain N nothing is bought and the noise is charged for anyway — which is what the sweep in ③ turns into a rule.

② The same four rows with the noise switched off

- This is the control experiment. If the rows differed here, the difference would be the filter's phase lag rather than noise amplification, and the argument would be about something else.

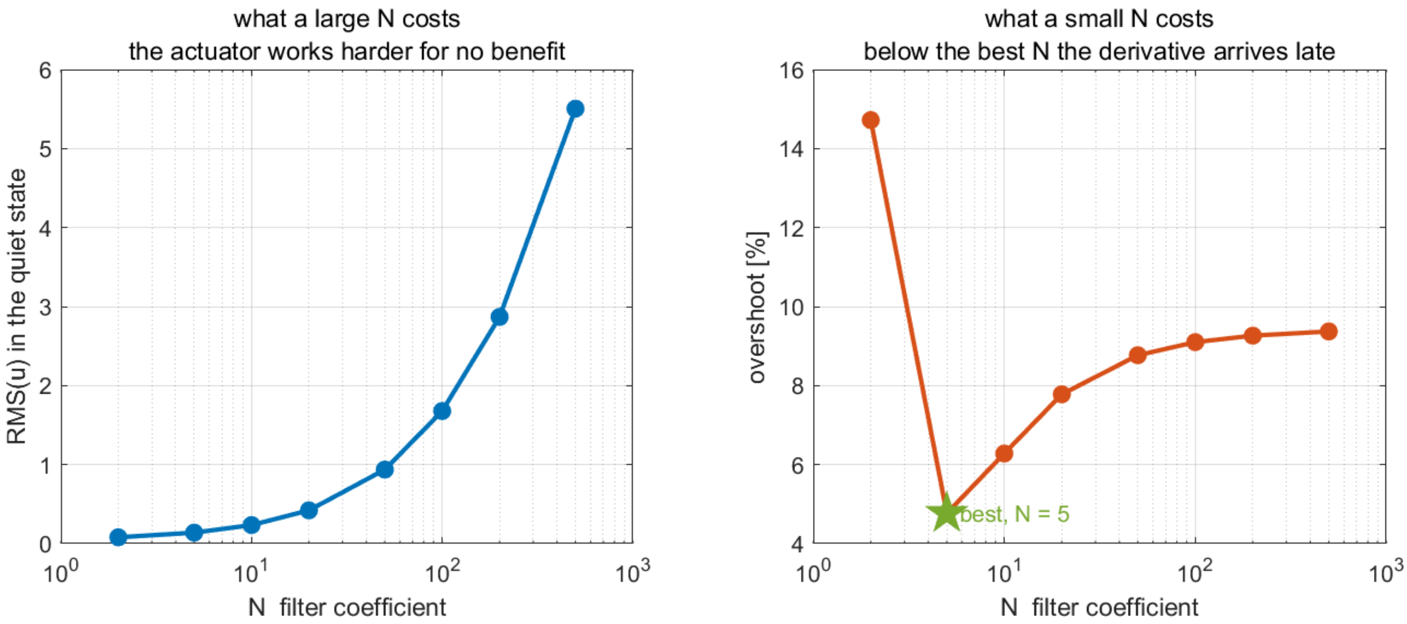
Derivative	overshoot	settling	RMS(u) quiet
P only	72.92%	19.00 s	0.176501
ideal, $K_d s$	9.47%	2.97 s	0.000002
pseudo, $N = 100$	9.13%	2.95 s	0.000001
pseudo, $N = 10$	6.19%	2.61 s	0.000000

- Every RMS collapses to zero. With no noise there is nothing for the derivative to amplify, so the factor of 37 between rows 2 and 4 in ① **was noise and nothing else**.
- The response changes by only 3.3 points of overshoot across rows 2 to 4, and the *slower* filter is the best of the three. Filtering the derivative is not a concession made under protest.

③ Sweeping N

N	overshoot	settling	RMS(u) quiet
2	14.72%	5.15 s	0.0800
5	$\mathbf{4.77\%}$	1.99 s	0.1389
10	6.28%	2.63 s	0.2348
20	7.78%	2.86 s	0.4212
50	8.77%	2.95 s	0.9370
100	9.10%	2.97 s	1.6780
200	9.26%	2.98 s	2.8691
500	9.37%	2.99 s	5.5044

W02 section I — above N ≈ 5 the response stops improving and only the noise grows



Reading the figure

Element	Meaning
left panel	actuator noise against N, on a logarithmic N axis — it never stops growing
right panel	overshoot against N, with the measured best value marked

What the figure says

- **Meaning.** The filter coefficient is swept over more than two decades and each run is reduced to two numbers: what it cost the actuator, and what it bought the response. The two panels are the price and the goods.
- **Trend, in numbers.** The left panel rises without ever turning: RMS(u) climbs from 0.08 at $N = 2$ to 5.50 at $N = 500$, a factor of 69, and the curve is steepest at the right-hand end. The right panel has a **minimum**: overshoot falls from 14.72% at $N = 2$ to 4.77% at $N = 5$, then rises again to 8.77% by $N = 50$ and **flattens** — from $N = 100$ to $N = 500$ it moves by 0.3 points.
- **Principle.** Below the closed-loop bandwidth the filter delays the derivative and damping is lost; above it, the filter is no longer shaping the response at all and is only deciding how much sensor noise reaches the actuator. **Those are two different mechanisms, and only one of them is a trade-off.**
- **Why the flattening is the important part.** A reader who sees two opposing curves concludes there is a trade to be tuned. There is — but only left of $N \approx 5$. Everywhere to the right the goods have stopped improving while the price keeps climbing, so **large N is not a cautious default; it is a purchase of nothing.**
- **What changes with the situation.** The best N tracks the closed-loop bandwidth, here $\omega_n = \sqrt{K_p} = 2.0$ rad/s against a measured optimum of 5. Retuning K_p moves the minimum; the shape of both curves does not change. That is what makes "place N just above the bandwidth" a rule rather than a coincidence of this plant.

> **★ Two regimes, not one trade-off**
>
> The two columns oppose each other only **below** $N \approx 5$. Above it, overshoot has flattened — from $N = 100$ to $N = 500$ it changes by 0.3 points — while RMS(u) grows by a factor of 3. Large N is not a safe default that merely costs a little noise; above the bandwidth it buys **nothing at all** and charges for it.
>
> The closed-loop bandwidth here is $\omega_n = \sqrt{K_p} = 2.0$ rad/s, and the measured best N is 5 — just above it. **Place N just above the closed-loop bandwidth and stop.**

Why this matters back on the vessel

- Week 3 controls heading, where the derivative term supplies genuine damping and is therefore indispensable. Its input is a yaw rate from an IMU, which is a noisy measurement.
- Everything in this section applies there unchanged, with one simplification: MSS convention feeds back the measured rate r directly instead of differentiating ψ. **The cleanest derivative is the one that was never taken** — if the state is already measured, no filter is needed and no noise is amplified.

Summary

Week Summary

Step	What was done	How it was verified
1	Identified the surge plant from a step response	K_u exact to four decimals; $\tau = 1.1200$ s against 1.1025 s predicted
2	Derived and measured the type 0 steady-state error	43.68, 13.43, 3.73 per cent at $K_p = 100$, 500, 2000, matching the final value theorem
3	Placed the closed-loop poles at $\zeta = 0.7$, $\omega_n = 1.5$	overshoot 8.15% measured against 8.07% predicted **with** the PI zero, 4.60% without it
4	Substituted the derivative term into the equation of motion	$\tau_{\text{eff}} = (M_{11} + K_d)/\lvert X_u \rvert$; overshoot predicted within 1.1 points over $K_d \in [0, 150]$
5	Established the exact speed ceiling	$u_{\max} = U_{\max} = 3.0864$ m/s, from $X_{\max} = 24.4g$ and $X_u = -24.4g/U_{\max}$
6	Measured windup and two remedies	integrator peak 3438 N; recovery 13.98 s against 3.40 s and 3.64 s
7	Compared the hand-built controller with the library PID block	identical to 2×10^{-16} m/s at $K_d = 0$; 0.18 m/s apart at $K_d = 60$
8	Isolated the principle on $1/(s + 1)$	integrator peak 60.00 against 1.225; recovery 32.04 s against 2.645 s
9	Swept the back-calculation gain	fastest recovery at $K_{\text{aw}} = 5$; undershoot $4.60 \to 43.96$ per cent across the sweep
10	Measured what the derivative filter buys	ideal derivative: actuator RMS 8.75 and peak 105.94; at $N = 10$, RMS 0.2348 with *less* overshoot
11	Swept the filter coefficient N	best overshoot at $N = 5$; above it overshoot flat within 0.3 points while RMS(u) grew $3\times$

Progress Check

Theory

- [ ] Able to state why the surge loop is type 0 without computing anything
- [ ] Able to derive $e_{ss}/u_d = 1/(1 + K_p K_u)$ from the final value theorem
- [ ] Able to invert ζ and ω_n for K_p and K_i, and to locate the PI zero
- [ ] Able to explain why K_d raises the overshoot on this axis and will not on the next

Laboratory

- [ ] `W02_0_setup` printed $\zeta = 0.7000$ and $\omega_n = 1.5000$
- [ ] each of `W02_C_…` through `W02_G_…` ran on its own and wrote its figure into `W02_simulink/img/`
- [ ] `W02_H_antiwindup_run` and `W02_I_pseudo_derivative_run` completed and wrote four more
- [ ] The open-loop identification was run and τ read from the 63.2% crossing

Recorded observations

- [ ] The measured steady-state error for three gains, against the predicted curve
- [ ] The overshoot with and without the PI zero in the prediction

- [ ] The integrator peak and the recovery time for all three anti-windup settings, with the tolerance band stated
- [ ] The gap between the hand-built controller and the library block, at $K_d = 0$ and $K_d = 60$
- [ ] The K_{extaw} at which recovery is fastest, and the undershoot it costs

---

Assignment 2

- **Due**: before the Week 3 session
- **Submit**: the modified `W02_0_setup.m`, a derivation, the numbers requested below, and two figures

① Requirements

1. Derive e_{ss}/u_d for the proportional loop from the final value theorem, showing the limit explicitly. Then determine, by hand, the gain $K_p^\star$ that leaves exactly 5% steady-state error at $u_d = 1.5$ m/s.
2. Determine the demanded steady force X_{ss} at $K_p^\star$ and state whether it lies inside the attainable control set of Appendix A1.
3. Design a PI controller for $\zeta = 1.0$ (critically damped poles) at $\omega_n = 1.2$ rad/s. State K_p, K_i, the pole locations and the zero location, and **predict the overshoot including the zero**. A critically damped pole pair with a zero does not give zero overshoot; say what it does give and why.
4. Determine, by hand, the largest step in u_d from rest that the design of ③ can command **without** the actuator saturating at any instant.

② Verification — mandatory

> **★ A claim is not a result. Numbers are required, with the tolerance band and the averaging window stated**

1. Run the proportional loop at $K_p^\star$ and report the measured steady-state error to four decimal places, with the averaging window stated.
2. Run the PI design of ①.3 and report the measured overshoot and 2% settling time. Compare with both predictions — poles only, and poles with the zero — and state which one the plant follows.
3. Apply the step of ①.4 and report the peak X_{cmd} and peak X_{sat}. State whether saturation occurred. Then apply a step 20% larger and repeat.
4. With the design of ①.3 and $u_d = 3.5$ m/s held for 35 s, sweep $K_{\text{aw}} \in \{0.2, 0.5, 1/\tau_u, 2, 5\}$ and report the recovery time for each. Produce **one figure** of recovery time against K_{aw}.
5. Produce **one figure** comparing the response of ①.3 with $K_d = 0$ and with $K_d = 40$, and report both overshoots.

③ Analysis (8–12 lines)

- Item ②.4 has a minimum somewhere in the sweep, or it does not. State which, and explain the behaviour at both ends: what happens as $K_{\text{aw}} \to 0$, and what happens when K_{aw} is made large. Relate the large-K_{aw} behaviour to the clamping scheme, which is the limiting case. Then state, in one sentence, what would remove the need for anti-windup entirely in this experiment.

Grading

Criterion	Weight
Derivations in ① are complete, including the overshoot prediction with the zero	20%
Verification performed and numbers reported with bands and windows stated	40%
Both figures are correct and readable	15%
Analysis in ③ explains both ends of the sweep and identifies the real remedy	25%

Troubleshooting

Symptom	Cause	Fix
The speed settles far below u_d with $K_i > 0$	`aw_mode = 2` with `Ki = 0` charges the integrator during saturation and can never discharge it	the model guards against this; if a modified version does not, set `aw_mode = 0` whenever `Ki = 0`
The integrator output is identically zero	an input of the `anti-windup` block is unconnected, so `Ki` arrives as 0	rebuild with `W02_1_build_surge_control`
The measured overshoot is far above the tabulated value for the design ζ	expected	a PI controller adds a zero at $-K_i/K_p$; compare against the linear model, not against the table
Adding K_d makes the response worse	expected on this axis	K_d adds to M_{11}; see §2-4
The vessel never reaches u_d however large the gain	u_d exceeds $u_{\max} = 3.0864$ m/s	no controller can pass this limit; reduce the setpoint
The response is unchanged when `Kp` is edited in the block dialog	the block reads the workspace variable	edit `W02_0_setup.m` and run it
`stepinfo` is undefined	Control System Toolbox is not licensed	the simulation results are unaffected; only the printed comparison in section E requires it
The library block and the hand-built path disagree at $K_d > 0$	expected	the block differentiates the error, the hand-built path the measurement; see §2-7
The library block's anti-windup setting cannot be changed from a variable	it is a dialog choice, not a signal	the hand-built path selects with `aw_mode`; the demonstration model of §H has one block per setting
`W02_H_antiwindup_run` reports a recovery equal to the run length	the run ended before the loop recovered	lengthen `aw_T`; the default 60 s is enough for $K_{\text{aw}} \geq 0.2$

References

Primary

- Fossen, T. I. *Handbook of Marine Craft Hydrodynamics and Motion Control*, 2nd ed. §12.2 (PID control of marine craft) and §12.2.6 (integrator anti-windup).

- Åström, K. J. and Hägglund, T. *Advanced PID Control*. ISA, 2006. Chapter 3, for the derivative filter and the two anti-windup schemes.
- MSS toolbox, `Tools/MSS/VESSELS/otter.m` — the damping and saturation constants, lines 65 and 91–98.

Course files

- `W02_simulink/W02_1_build_surge_control.m` — the model generator
- `W02_simulink/W02_0_setup.m` — the parameters, including the PI design equations
- `W02_simulink/W02_C_identify_plant.m` … `W02_G_block_vs_handbuilt.m` — one script per lecture section
- `W02_simulink/W02_plot.m` — the summary figure, called by both the runner and the model's `StopFcn`
- `W02_simulink/W02_animate.m` — the live view
- `W02_simulink/W02_H_build_antiwindup.m` — the standalone demonstration model of §H
- `W02_simulink/W02_H_antiwindup_run.m` , `W02_aw_plot.m` — section H, its experiments and figures

Next Week

- **Week 3 — Heading Control**
- The heading axis contains a free integrator, $\psi = \int r$, so the loop is **type 1** and proportional action alone leaves no steady-state error. Everything §2-2 established is reversed by one structural change.
- The derivative term reappears, this time acting on the yaw rate, and this time it damps.
- The angle ψ wraps, and the smallest-signed-angle function `ssa` is what keeps a controller from steering the long way round.
- Preparation: bring $M_{66} = 42.65$ kg·m² and $N_r = -42.65$ from Week 1, and the finding of §2-4 about where a derivative term goes.